EXAM NOTES · ฉบับละเอียด

Computer Architecture + Digital Logic

สรุปเตรียมสอบ อธิบายทุกคำศัพท์: ย่อมาจากอะไร คืออะไร ทำหน้าที่อะไร ทำงานอย่างไร และเกี่ยวข้องกับอะไร

ระบบสอบ	MyCourseVille (สอบ on-site)
เวลา	90 นาที (10:00–11:30)
คะแนน	เต็ม 60 → คิดเป็นคะแนนเก็บ 20
Choice	40 ข้อ × 1 คะแนน (ผิดไม่ติดลบ)
ตอบสั้น	10 ข้อ × 2 คะแนน
สัดส่วน	Digital Logic 15 · Computer Architecture 45

ส่วน Computer Architecture อ้างอิงสไลด์ *Computer Architecture: Design and Analysis* (Krerk Piromsopa, Ph.D.) ครบ 397 หน้า · ส่วน Digital Logic อ้างอิงเนื้อหามาตรฐานของวิชา เพราะไฟล์ PDF ที่ได้รับไม่มีสไลด์ส่วนนี้

สารบัญ

วิธีใช้เอกสารนี้

รูปแบบในเอกสาร
กลยุทธ์ทำข้อสอบ 90 นาที
แผนที่ภาพรวมของวิชา

Part 0: Digital Logic (15 คะแนน)

0.1 ระบบเลข (Number Systems)
0.2 การแทนเลขมีเครื่องหมาย (Signed Numbers)
0.3 Logic Gates และ Boolean Algebra
0.4 Canonical Forms และ K-map
0.5 Combinational Circuits (วงจรเชิงผสม)
0.6 Sequential Circuits (วงจรเชิงลำดับ)
0.7 สรุปกับดัก Digital Logic

Ch1: Introduction to Computer Architecture

1.1 ภาพรวม: วิชานี้เรียนอะไร
1.2 Design Trade-offs และ Goals
1.3 ทำไมต้องเรียน และ How to design

Ch2: Performance: Measurement and Analysis

2.1 ภาพรวม
2.2 การวัด: Response Time vs Throughput
2.3 Benchmarks
2.4 การรวมผลหลายโปรแกรม: Mean
2.5 CPU Time Equation
2.6 Amdahl's Law

Ch3: Instruction Set Architecture

3.1 ภาพรวม
3.2 Design Issues และคุณสมบัติของ ISA ที่ดี
3.3 ISA 5 แบบ (ตามตำแหน่งของ operand)
3.4 Memory Organization
3.5 Addressing Modes
3.6 ประเภทคำสั่งและ Control Flow
3.7 Exceptions และ Interrupts
3.8 Instruction Format
3.9 Compiler และ Optimization

Ch4: Single Cycle Processor (nanoLADA)

4.1 ภาพรวม
4.2 Design Steps (5 ขั้น)

4.3 nanoLADA ISA
4.4 Components ของ Datapath
4.5 Control Signals
4.6 Critical Path

Ch5: Multiple Cycle Processor

5.1 ภาพรวม
5.2 วิธีลด Cycle Time
5.3 Physical RTL และจำนวน Cycle
5.4 Control แบบ FSM และ Microprogram
5.5 Exercises Ch5

Ch6: Pipelining

6.1 ภาพรวม
6.2 Speedup ของ Pipeline
6.3 Hazards
6.4 Data Hazard 3 แบบ (Part 2)
6.5 Pipelined Datapath, Hazard Detection, Forwarding
6.6 Exercises Ch6

Ch7: Contemporary Architecture (ILP)

7.1 ภาพรวม
7.2 Vector Processor (SIMD)
7.3 Superscalar, Superpipeline, VLIW
7.4 Compiler & ILP
7.5 Multicore และ Thread-Level Parallelism

Ch8: Memory Organization

8.1 ภาพรวม: ทำไมต้องมี Memory Hierarchy
8.2 Memory Technology
8.3 Principle of Locality
8.4 Cache: คำศัพท์พื้นฐาน
8.5 Direct-Mapped Cache: Tag, Index, Offset
8.6 Cache Performance
8.7 Sources of Miss (3C)
8.8 Cache Configurations (5 อย่าง)
8.9 Write Management
8.10 Software & Cache
8.11 Virtual Memory (Ch8 part 2)
8.12 Overlapped Cache & TLB Access

Ch9: Storage Architecture

9.1 Storage และ Interface
9.2 RAID

9.3 Network Storage

Formula Sheet

กับดักรวม (อ่านก่อนเข้าห้องสอบ)

Glossary A–Z (ค้นหาเร็ว)

วิธีใช้เอกสารนี้

รูปแบบในเอกสาร

เอกสารนี้ใช้กล่อง 5 แบบ ให้ดูสีแล้วรู้ทันทีว่ากล่องนั้นเป็นอะไร

คำศัพท์ *Full English name* ชื่อภาษาไทย

- **คืออะไร:** นิยามสั้นที่สุด อ่านแล้วรู้ทันที
- **ทำหน้าที่:** มีไว้เพื่ออะไร แก้ปัญหาอะไร
- **ทำงานอย่างไร:** กลไกหรือขั้นตอน
- **สัมพันธ์กับ:** คำศัพท์อื่นที่เกี่ยวข้อง และบทที่เกี่ยวข้อง

★ **จำให้ได้** สูตรหรือข้อเท็จจริงที่ต้องจำให้ได้ในห้องสอบ

🔗 **ความสัมพันธ์** เชื่อมโยงเรื่องนี้กับเรื่องอื่น ใช้ตอบคำถามแบบ "ทำไม" และ "มีผลต่ออะไร"

✎ **ตัวอย่าง** ตัวอย่างพร้อมวิธีคิด ส่วนใหญ่มาจากสไลด์โดยตรง

⚠ **กับดัก** จุดที่ทำผิดบ่อย และตัวเลือกหลอกที่มักเจอในข้อสอบ Choice

กลยุทธ์ทำข้อสอบ 90 นาที

ช่วงเวลา	ทำอะไร
0–40 นาที	Choice 40 ข้อ ข้อละไม่เกิน 1 นาที ข้อที่ต้องคำนวณยาวให้ข้ามไปก่อน
40–80 นาที	ตอบสั้น 10 ข้อ ข้อละประมาณ 4 นาที พิมพ์ตัวเลข**พร้อมหน่วย**
80–90 นาที	ย้อนทำข้อที่ข้าม และ**เดาข้อที่ยังว่างให้หมด** (ผิดไม่ติดลบ)

แผนที่ภาพรวมของวิชา

วิชานี้ทั้งหมดเป็นเรื่องเดียวกัน คือ **ทำให้โปรแกรมทำงานเร็วที่สุดภายใต้ cost และ power ที่จำกัด** โดยทุกบทไปกระทบตัวแปรใดตัวแปรหนึ่งใน **CPU Time = IC × CPI × T_c**

ชั้น	บท	ส่งผลต่อ
โปรแกรม + Compiler	Ch3 (optimization), Ch7 (loop unroll)	**IC** (จำนวนคำสั่ง) และการ schedule
ISA (ชุดคำสั่ง)	Ch3	**IC** และ **CPI**
Datapath + Control	Ch4 (single cycle), Ch5 (multi cycle)	**CPI** และ **T_c** (cycle time)
Pipelining / ILP	Ch6, Ch7	**CPI** ที่มีผล (ทำหลายคำสั่งซ้อนกัน)

ชั้น	บท	ส่งผลต่อ
Memory hierarchy	Ch8	**CPI** (stall รอ memory)
Storage	Ch9	เวลา I/O (นอก CPU time)
Digital Logic	Part 0	วงจรพื้นฐาน (gate, MUX, adder, flip-flop, FSM) ที่ใช้สร้างทุกอย่างข้างบน

🔗 **ความสัมพันธ์** อ่านจากล่างขึ้นบน: **gate → MUX/adder/flip-flop → register file/ALU → datapath + control (FSM) → pipeline → cache/virtual memory → storage** แต่ละชั้นใช้ชั้นก่อนหน้าเป็นชิ้นส่วน ถ้าโจทย์ถามว่า "X มีผลต่ออะไร" ให้ไล่ดูว่า X ไปเปลี่ยน IC, CPI หรือ T_c

Part 0: Digital Logic (15 คะแนน)

> ไฟล์ PDF ที่ได้รับไม่มีสไลด์ Digital Logic ส่วนนี้จึงอิงเนื้อหามาตรฐานของวิชา Digital Computer Logic (สไลด์ Com Arch อ้างถึงหนังสือ *Contemporary Logic Design* ของ Randy Katz) และเน้นหัวข้อที่ Com Arch นำไปใช้ต่อ

0.1 ระบบเลข (Number Systems)

Bit *Binary digit* เลขฐานสองหนึ่งหลัก

- **คืออะไร:** หน่วยข้อมูลที่เล็กที่สุด มีค่าได้แค่ 0 หรือ 1
- **ทำงานอย่างไร:** ในวงจรแทนด้วยแรงดันต่ำ (0) หรือสูง (1)
- **สัมพันธ์กับ:** 8 bits = 1 **byte** · 4 bits = 1 **nibble** (= เลขฐาน 16 หนึ่งหลัก) · n bits แทนค่าได้ **2^n แบบ**

Radix / Base ฐานของระบบเลข

- **คืออะไร:** จำนวนสัญลักษณ์ที่ใช้ในระบบเลข เช่น ฐาน 2 (binary), 8 (octal), 10 (decimal), 16 (hexadecimal)
- **ทำงานอย่างไร:** หลักที่ i มีค่าประจำหลักเป็น base^i เช่น $(1011.01)_2 = 1{\cdot}2^3 + 0{\cdot}2^2 + 1{\cdot}2^1 + 1{\cdot}2^0 + 0{\cdot}2^{-1} + 1{\cdot}2^{-2} =$ **11.25**
- **สัมพันธ์กับ: Hex** ใช้แทน binary แบบสั้น (1 hex = 4 bits) ซึ่ง address และ opcode ในวิชา Com Arch เขียนเป็น hex เสมอ

วิธีแปลงฐาน

จาก → ไป	วิธี	ตัวอย่าง
ฐานใดก็ได้ → ฐาน 10	คูณแต่ละหลักด้วยค่าประจำหลักแล้วรวม	$(1011.01)_2 = 11.25$
ฐาน 10 → ฐาน 2 (จำนวนเต็ม)	หาร 2 ซ้ำ ๆ แล้ว**อ่านเศษจากล่างขึ้นบน**	13 → 1101
ฐาน 10 → ฐาน 2 (ทศนิยม)	คูณ 2 ซ้ำ ๆ แล้ว**อ่านหลักหน้าจุดจากบนลงล่าง**	0.375 → .011
ฐาน 2 ↔ ฐาน 16	จัดกลุ่มทีละ **4 bits** เริ่มจากจุด	1111 0111 = 0xF7
ฐาน 2 ↔ ฐาน 8	จัดกลุ่มทีละ **3 bits** เริ่มจากจุด	0x3F7 = 001 111 110 111 = $\mathbf{1767_8}$

Dec	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

ค่ายกกำลังของ 2 ที่ควรจำ: $2^{10} = 1024$ (K), $2^{20} \approx 1$M, $2^{30} \approx 1$G, $2^{32} = 4$G

0.2 การแทนเลขมีเครื่องหมาย (Signed Numbers)

MSB / LSB *Most / Least Significant Bit* บิตที่มีค่าประจำหลักมากที่สุด / น้อยที่สุด

- **คืออะไร:** MSB คือบิตซ้ายสุด ส่วน LSB คือบิตขวาสุด
- **ทำหน้าที่:** ในเลขมีเครื่องหมาย MSB เป็น **sign bit** (0 = บวก, 1 = ลบ)
- **สัมพันธ์กับ: Endian** (Ch3) คือการเลือกว่าจะเก็บ byte ที่มี MSB หรือ LSB ไว้ที่ address ต่ำ

Sign-magnitude เครื่องหมายและขนาด

- **คืออะไร:** ใช้ MSB เป็นเครื่องหมาย ส่วนบิตที่เหลือเป็นขนาดของตัวเลข เช่น –5 (8 bits) = 1000 0101
- **ข้อเสีย:** มีศูนย์ 2 ตัว (+0, –0) และวงจรบวก/ลบซับซ้อน

1's complement *One's complement* ส่วนเติมเต็มหนึ่ง

- **คืออะไร:** ทำค่าลบโดย**กลับทุกบิต** เช่น –5 = 1111 1010
- **ข้อเสีย:** ยังมีศูนย์ 2 ตัว และการบวกต้องเอา carry วนกลับมาบวก (end-around carry)

2's complement *Two's complement* ส่วนเติมเต็มสอง

- **คืออะไร:** วิธีแทนเลขลบที่ใช้ในคอมพิวเตอร์ปัจจุบันเกือบทั้งหมด
- **ทำงานอย่างไร:** –X = **กลับทุกบิตของ X แล้วบวก 1**
  - ทางลัด: เก็บบิตจากขวาไว้จนถึงเลข 1 ตัวแรก แล้วกลับบิตที่เหลือทั้งหมด
  - ตัวอย่าง: +100 = 0110 0100 → 1001 1011 → +1 = **1001 1100 = 0x9C = –100**
- **ข้อดี:** มี**ศูนย์ตัวเดียว** และ**ใช้ adder ตัวเดียวกันได้ทั้งบวกและลบ** ($A - B = A + \bar{B} + 1$)
- **ช่วงค่า (n bits): -2^{n-1} ถึง $2^{n-1} - 1$** ซึ่ง**ไม่สมมาตร** เช่น 8 bits ได้ –128 ถึง +127
- **สัมพันธ์กับ:** ALU ของ nanoLADA (Ch4) ทำ SUB ด้วยวิธีนี้ ส่วน **sign extension** ใช้ขยายจำนวนบิต

แบบ (n bits)	ช่วงค่า	ศูนย์	8-bit ของ –5
Unsigned	0 ถึง $2^n - 1$	1 ตัว	—
Sign-magnitude	$-(2^{n-1} - 1)$ ถึง $2^{n-1} - 1$	2 ตัว	1000 0101
1's complement	$-(2^{n-1} - 1)$ ถึง $2^{n-1} - 1$	2 ตัว	1111 1010
2's complement	**-2^{n-1} ถึง $2^{n-1} - 1$**	**1 ตัว**	**1111 1011**

Sign extension การขยายเครื่องหมาย

- **คืออะไร:** ขยายเลข 2's complement ให้มีบิตมากขึ้นโดยค่าไม่เปลี่ยน
- **ทำงานอย่างไร: คัดลอก sign bit** ไปเติมทางซ้าย เช่น 1001 (–7) → 1111 1001 (–7)
- **สัมพันธ์กับ:** `sign_ext(imm)` ใน nanoLADA (LW/SW/BEQ) ขยาย immediate 16 บิตเป็น 32 บิต ส่วน **zero extension** เติม 0 ใช้กับเลข unsigned

Carry vs Overflow บิตทด vs ผลล้นช่วง

- **Carry out:** บิตที่ทดออกจากหลักซ้ายสุด ซึ่ง**มีความหมายกับเลข unsigned** (ถ้าเป็น 1 แปลว่าผลเกิน $2^n - 1$)
- **Overflow (signed):** ผลลัพธ์จริงอยู่นอกช่วงที่ n bits แทนได้
  - เกิดเมื่อ **บวก + บวก ได้ลบ** หรือ **ลบ + ลบ ได้บวก**
  - **บวกเลขต่างเครื่องหมายไม่มีทาง overflow**
  - ฮาร์ดแวร์ตรวจด้วย: **overflow = Cin ของ MSB ⊕ Cout ของ MSB**

✎ **ตัวอย่าง** 8 bits: 0x70 + 0x50 = 0111 0000 + 0101 0000 = **1100 0000**

- **carry out = 0**
- แต่บวก + บวก ได้ลบ → **overflow = 1** (112 + 80 = 192 เกิน 127)

8 bits: 0xF0 + 0x20 = (−16) + 32 → ได้ 1 0001 0000

- **carry out = 1**
- ผล 0x10 = 16 ถูกต้อง → **ไม่ overflow**

BCD *Binary-Coded Decimal* เลขฐานสิบเข้ารหัสฐานสอง

- **คืออะไร:** ใช้ 4 bits แทนเลขฐานสิบ 1 หลัก (0–9) เช่น 59 = 0101 1001
- **ข้อสังเกต:** รหัส 1010–1111 ไม่ได้ใช้ จึงเปลืองบิตกว่า binary ปกติ

Gray code รหัสเกรย์

- **คืออะไร:** รหัสที่ค่าติดกัน**ต่างกันเพียง 1 บิต** เช่นลำดับ 2 บิต: 00, 01, 11, 10
- **ทำงานอย่างไร:** binary → Gray ด้วย g = b ⊕ (b >> 1)
- **สัมพันธ์กับ:** แกนของ **K-map** เรียงตาม Gray code เพื่อให้ช่องที่ติดกันต่างกันแค่ 1 ตัวแปร

0.3 Logic Gates และ Boolean Algebra

Gate	สัญลักษณ์ทางพีชคณิต	Output เป็น 1 เมื่อ
NOT	A'	A = 0
AND	A·B	ทุกอินพุตเป็น 1
OR	A + B	มีอินพุตอย่างน้อยหนึ่งตัวเป็น 1
NAND	(A·B)'	ไม่ใช่ทุกตัวเป็น 1
NOR	(A + B)'	ทุกตัวเป็น 0
XOR	A ⊕ B = A'B + AB'	อินพุต**ต่างกัน** (หรือจำนวน 1 เป็นเลขคี่)
XNOR	(A ⊕ B)'	อินพุต**เท่ากัน** ใช้ทำ equality comparator

Universal gate เกตครอบจักรวาล

- **คืออะไร:** gate ชนิดเดียวที่ใช้สร้างฟังก์ชันใดก็ได้ ได้แก่ **NAND** และ **NOR**
- **ทำงานอย่างไร:**
  - NOT A = A NAND A
  - A·B = (A NAND B)' (NAND แล้วกลับด้วย NAND อีกตัว)
  - A + B = A' NAND B'
- **ทำไมสำคัญ:** โรงงานผลิต IC สร้าง gate ชนิดเดียวได้ถูกและเร็ว

กฎของ Boolean Algebra

กฎ	แบบ AND	แบบ OR
Identity	A·1 = A	A + 0 = A
Null	A·0 = 0	A + 1 = 1
Idempotent	A·A = A	A + A = A
Complement	A·A' = 0	A + A' = 1
Absorption	A(A + B) = A	A + AB = A
ลดรูป (สำคัญ)	A(A' + B) = AB	**A + A'B = A + B**
DeMorgan	**(AB)' = A' + B'**	**(A + B)' = A'B'**
Distributive	A(B + C) = AB + AC	**A + BC = (A + B)(A + C)**
Consensus	AB + A'C + BC = AB + A'C	

★ จำให้ได้ **DeMorgan: "กลับขีดแล้วสลับเครื่องหมาย"** เติมขีดบนให้แต่ละตัวแปร แล้วเปลี่ยน AND ↔ OR
Duality: สลับ AND ↔ OR และ 0 ↔ 1 แล้วสมการยังเป็นจริง

0.4 Canonical Forms และ K-map

Minterm (m_i) ผลคูณมาตรฐาน

- **คืออะไร:** AND ของ**ทุกตัวแปร** (ตัวแปรละครั้ง ในรูปปกติหรือกลับ) ที่ให้ค่า 1 **เพียงแถวเดียว** (แถว i) ของ truth table
- **ทำงานอย่างไร:** แถวไหนบิตเป็น 0 ให้ใช้ตัวกลับ เช่น 3 ตัวแปร m_5 = 101 = **AB'C**
- **สัมพันธ์กับ: SOP** F = Σm(แถวที่ F = 1) และ **decoder** (output แต่ละเส้นของ decoder คือ minterm หนึ่งตัว)

Maxterm (M_i) ผลบวกมาตรฐาน

- **คืออะไร:** OR ของทุกตัวแปรที่ให้ค่า 0 **เพียงแถวเดียว** (แถว i)
- **ทำงานอย่างไร:** แถวไหนบิตเป็น 1 ให้ใช้ตัวกลับ เช่น M_5 = 101 = **A' + B + C'**
- **สัมพันธ์กับ: POS** F = ΠM(แถวที่ F = 0)

SOP / POS *Sum of Products / Product of Sums*

- **SOP:** OR ของ AND terms เช่น F = AB + C' (สร้างจาก**แถวที่เป็น 1**)
- **POS:** AND ของ OR terms เช่น F = (A + B)(C') (สร้างจาก**แถวที่เป็น 0**)
- **ความสัมพันธ์:** Σm กับ ΠM ของฟังก์ชันเดียวกันใช้**เลขแถวที่เหลือกันพอดี** เช่น 3 ตัวแปร Σm(1,3,5) = ΠM(0,2,4,6,7)
- n ตัวแปรมี truth table 2^n แถว และมีฟังก์ชันที่เป็นไปได้ 2^(2^n) ฟังก์ชัน

K-map *Karnaugh map* แผนที่คาร์นอห์

- **คืออะไร:** ตาราง truth table ที่จัดช่องให้ช่องติดกันต่างกันเพียง 1 ตัวแปร เพื่อใช้ลดรูปสมการด้วยสายตา
- **ทำงานอย่างไร:**
  1. เรียงแกนแบบ **Gray code 00, 01, 11, 10** (ไม่ใช่ 00, 01, 10, 11)
  2. ขอบซ้ายติดกับขอบขวา และขอบบนติดกับขอบล่าง (**wrap around**) ดังนั้น**มุมทั้งสี่ติดกัน**
  3. วงกลุ่มเลข 1 ขนาด 2^k (1, 2, 4, 8, 16) ให้**กลุ่มใหญ่ที่สุดและจำนวนกลุ่มน้อยที่สุด** กลุ่มซ้อนกันได้
  4. ตัวแปรที่ค่าเปลี่ยนภายในกลุ่มจะหายไป
- **สัมพันธ์กับ: Don't care**, **prime implicant**, การสร้าง control logic ของ processor

ตำแหน่งเลข minterm ใน K-map 4 ตัวแปร (แถวเป็น AB และคอลัมน์เป็น CD):

AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

Don't care (X หรือ d) ค่าที่ไม่สนใจ

- **คืออะไร:** แถวที่อินพุตนั้นไม่มีทางเกิดขึ้น หรือ output เป็นอะไรก็ได้ (เช่น BCD 1010–1111)
- **ทำงานอย่างไร:** ใน K-map **จะนับเป็น 1 ก็ได้ถ้าช่วยให้กลุ่มใหญ่ขึ้น** หรือ**ไม่ต้องวงก็ได้** (ไม่บังคับให้ครอบ)
- **สัมพันธ์กับ:** ค่า `x` ในตาราง control signals ของ nanoLADA ก็คือ don't care

Prime Implicant / Essential PI กลุ่มที่ขยายต่อไม่ได้ / กลุ่มที่ต้องเลือก

- **Prime implicant:** กลุ่มที่ใหญ่ที่สุดแล้ว ขยายต่อไม่ได้อีก
- **Essential prime implicant:** PI ที่ครอบเลข 1 บางตัวซึ่ง PI อื่นไม่ครอบ จึง**ต้องอยู่ในคำตอบเสมอ**

- **Σm(0,2,8,10)** คือ 4 มุม → **B'D'**
- **Σm(0,2,8,10) + d(1,3,9,11)** ครอบทุกช่องที่ B = 0 → **B'** (กลุ่ม 8 ช่องจึงดีกว่า)
- **3 ตัวแปร Σm(0,1,4,5)** ทุกตัวมี B = 0 → **B'**

0.5 Combinational Circuits (วงจรเชิงผสม)

Combinational circuit วงจรเชิงผสม

- **คืออะไร:** วงจรที่ output **ขึ้นกับ input ปัจจุบันเท่านั้น** ไม่มีหน่วยความจำ
- **ตัวอย่าง:** MUX, decoder, adder, ALU, comparator
- **สัมพันธ์กับ:** single-cycle processor (Ch4) ทำทุกอย่างของคำสั่งเป็น combinational logic ภายใน 1 cycle ซึ่งตรงข้ามกับ **sequential circuit**

MUX *Multiplexer* วงจรเลือกข้อมูล

- **คืออะไร:** วงจรที่**เลือก 1 อินพุตจากหลายอินพุต**ส่งออกไปทาง output
- **ทำงานอย่างไร:** MUX **2^n-to-1** มี **select lines n เส้น** เช่น 32-to-1 มี select 5 บิต
  - สมการของ 2-to-1: $Y = S' \cdot I_0 + S \cdot I_1$
- **ใช้สร้างฟังก์ชัน:** MUX 2^n-to-1 สร้างฟังก์ชัน n ตัวแปรใดก็ได้ (ต่อค่า truth table เข้าที่ data inputs)
- **สัมพันธ์กับ:** ใน datapath ทุกจุดที่ต้อง "เลือก" ใช้ MUX ทั้งหมด เช่น `sel_b` (เลือก R[rt] หรือ imm), `sel_data` (เลือกผล ALU หรือ memory), `sel_wr` (เลือก rd หรือ rt) และ forwarding MUX ใน pipeline

DEMUX *Demultiplexer*

- **คืออะไร:** ส่งอินพุตเดียวออกไปยัง output 1 จาก 2^n เส้นตาม select (ทำงานกลับด้านกับ MUX)

Decoder วงจรถอดรหัส

- **คืออะไร:** รับอินพุต n บิตแล้วทำให้ output **1 เส้นจาก 2^n เส้น** เป็น 1 (**one-hot**)
- **ทำงานอย่างไร:** output เส้นที่ i = minterm m_i จึงเอา **decoder + OR gate** สร้าง SOP ใดก็ได้
- **สัมพันธ์กับ: register file** ใช้ decoder 5-to-32 เลือก register ที่จะเขียน และ control unit ใช้ decode opcode

Encoder / Priority encoder วงจรเข้ารหัส

- **Encoder:** ทำงานกลับด้านกับ decoder คือแปลงอินพุต one-hot 2^n เส้นเป็นรหัส n บิต
- **Priority encoder:** ถ้ามีหลายอินพุตเป็น 1 พร้อมกัน จะเลือกตัวที่ **priority สูงสุด** (ใช้กับ interrupt หลายแหล่ง)

Comparator วงจรเปรียบเทียบ

- **ทำงานอย่างไร:** เทียบว่าเท่ากันไหมด้วย **XNOR ทุกบิตแล้ว AND รวม**
- **สัมพันธ์กับ:** ใช้เทียบ **tag ใน cache** (n-way ใช้ n comparators) และใน **hazard detection** (rs == rw_ex) ส่วน BEQ ใน nanoLADA ใช้ SUB แล้วดู zero flag

Adders

Half Adder (HA) วงจรบวกครึ่ง

- **คืออะไร:** บวกเลข 1 บิตสองตัว **ไม่มี carry-in**
- **สมการ: $S = A \oplus B$** และ **$C = A \cdot B$**

Full Adder (FA) วงจรบวกเต็ม

- **คืออะไร:** บวก A + B + **Cin** (มี carry-in) เป็นหน่วยพื้นฐานของการบวกหลายบิต
- **สมการ: S = A ⊕ B ⊕ Cin** และ **Cout = AB + Cin(A ⊕ B)** = AB + ACin + BCin
- **สัมพันธ์กับ:** สร้างจาก HA 2 ตัว + OR gate และนำไปต่อเป็น ripple-carry adder

RCA *Ripple-Carry Adder* วงจรบวกแบบทดต่อเนื่อง

- **คืออะไร:** ต่อ FA n ตัวเป็นลูกโซ่ โดย Cout ของบิต i เป็น Cin ของบิต i + 1
- **ข้อเสีย:** carry ต้อง "ไหล" ผ่านทุกบิต delay จึง**แปรตาม n** (เช่น FA ละ 2 gate delays → 16 bits ≈ 32 gate delays)

CLA *Carry-Lookahead Adder* วงจรบวกแบบมองล่วงหน้า

- **คืออะไร:** adder ที่คำนวณ carry ทุกบิต**พร้อมกัน** ไม่ต้องรอ carry ไหลทีละบิต
- **ทำงานอย่างไร:**
  - **Generate $g_i = A_iB_i$** (บิตนี้สร้าง carry เอง)
  - **Propagate $p_i = A_i \oplus B_i$** (บิตนี้ส่ง carry ต่อ)
  - $c_{i+1} = g_i + p_ic_i$ แล้วกระจายสมการให้เหลือแค่ g, p และ c_0
- **Trade-off: เร็วกว่าแต่ใช้ gate มากกว่า** (area vs speed)

ALU *Arithmetic Logic Unit* หน่วยคำนวณเลขคณิตและตรรกะ

- **คืออะไร:** วงจร combinational ที่ทำ operation ทางคณิตศาสตร์ (+, –) และตรรกะ (AND, OR) ตาม **control (alu_ops)**
- **ทำงานอย่างไร:**
  1. ภายในมี adder และ logic unit คำนวณทุกแบบพร้อมกัน แล้ว **MUX เลือกผล**ตาม alu_ops
  2. การลบใช้ **$\bar{B}$ และ Cin = 1** (2's complement)
  3. **Zero flag = NOR ของทุกบิตของผลลัพธ์** (เป็น 1 เมื่อผลเป็น 0)
- **สัมพันธ์กับ:** nanoLADA ALU มี `00` ADD, `01` OR, `10` SUB และใช้ zero flag กับ BEQ (Ch4) ส่วนในสไลด์ ALU เป็นส่วนที่ช้าที่สุดของ critical path (60 ns)

Programmable Logic

ROM *Read-Only Memory* หน่วยความจำอ่านอย่างเดียว

- **ในบริบทวงจร:** ใช้เก็บ truth table ทั้งตาราง โดย address = input และ data = output (2^n แถว)
- **สัมพันธ์กับ:** ใช้ implement control unit และ **microprogram (Map ROM)** ใน Ch5

PLA / PAL *Programmable Logic Array / Programmable Array Logic*

- **PLA:** มี **AND plane** (สร้าง product terms) และ **OR plane** (รวม terms) ที่ **program ได้ทั้งคู่** จึงประหยัดกว่า ROM เมื่อ product term ใช้ร่วมกันได้
- **PAL:** AND plane program ได้ แต่ **OR plane fixed**
- **สัมพันธ์กับ:** สไลด์ Ch4 สร้าง control ของ nanoLADA ด้วย PLA (G1–G7 คือ AND term ของ opcode แต่ละตัว)

Timing

Propagation delay (t_{pd}) เวลาหน่วงการแพร่สัญญาณ

- **คืออะไร:** เวลาตั้งแต่ input เปลี่ยนจนถึง output นิ่ง
- **สัมพันธ์กับ: critical path** คือ path ที่ delay รวมมากที่สุด ซึ่งเป็นตัวกำหนด clock period (เทียบกับ nanoLADA ที่ใช้ 230 ns)

Glitch / Hazard (ในวงจร) สัญญาณกระพริบ

- **คืออะไร:** output เปลี่ยนค่าชั่วคราวผิดพลาด เพราะ path ต่าง ๆ มี delay ไม่เท่ากัน
- **วิธีแก้:** static-1 hazard แก้ได้ด้วยการเพิ่ม consensus term
- **อย่าสับสน:** คำว่า **hazard ใน pipeline (Ch6)** เป็นคนละเรื่องกัน

0.6 Sequential Circuits (วงจรเชิงลำดับ)

Sequential circuit วงจรเชิงลำดับ

- **คืออะไร:** วงจรที่ output ขึ้นกับ **input + state (ค่าที่จำไว้)** โดยมีหน่วยความจำ (latch/flip-flop)
- **Synchronous:** state เปลี่ยนเฉพาะที่ขอบ **clock**
- **สัมพันธ์กับ:** register, counter, FSM และ control unit ของ multi-cycle processor (Ch5)

Clock สัญญาณนาฬิกา

- **คืออะไร:** สัญญาณสี่เหลี่ยมที่เป็นจังหวะให้วงจร synchronous
- **ค่าที่เกี่ยวข้อง: Clock rate (f)** = จำนวน cycle ต่อวินาที (Hz) และ **Cycle time T_c = 1/f** เช่น 2 GHz → 0.5 ns
- **สัมพันธ์กับ:** CPU Time = IC × CPI × T_c (Ch2)

Latch vs Flip-flop (FF) แลตช์ vs ฟลิปฟล็อป

- **Latch: level-sensitive** (โปร่งใส ส่งค่าผ่านตลอดช่วงที่ enable = 1)
- **Flip-flop: edge-triggered** (รับค่าเฉพาะขอบขาขึ้นหรือขาลงของ clock)
- **สัมพันธ์กับ:** register file (32 × 32 D-FF) และ **pipeline registers** ระหว่าง stage

FF	Characteristic equation	พฤติกรรม
D (Data/Delay)	**$Q^+ = D$**	เก็บค่า D ใช้ทำ register

FF	Characteristic equation	พฤติกรรม
JK	**$Q^+ = JQ' + K'Q$**	00 = hold, 01 = reset (0), 10 = set (1), **11 = toggle**
T (Toggle)	**$Q^+ = T \oplus Q$**	T = 1 → กลับค่า ใช้ทำ counter
SR (Set-Reset)	$Q^+ = S + R'Q$ (ต้องมี $S \cdot R = 0$)	**S = R = 1 ห้ามใช้**

Excitation table (รู้ว่าอยากให้ Q เปลี่ยนเป็นอะไร แล้วต้องป้อนอะไร):

Q → Q^+	D	T	J K	S R
0 → 0	0	0	0 X	0 X
0 → 1	1	1	1 X	1 0
1 → 0	0	1	X 1	0 1
1 → 1	1	0	X 0	X 0

Setup time / Hold time เวลาตั้งค่า / เวลาคงค่า

- **Setup time:** เวลาที่ input ต้องนิ่ง**ก่อน**ขอบ clock
- **Hold time:** เวลาที่ input ต้องนิ่ง**หลัง**ขอบ clock
- **Clock ขั้นต่ำ:** $T_{clk} \geq t_{clk\text{-}to\text{-}Q} + t_{comb,max} + t_{setup}$
- **สัมพันธ์กับ:** เป็นเหตุผลที่ pipeline register มี **overhead** ทำให้ pipeline เร็วขึ้นได้ไม่ถึง k เท่าพอดี (Ch6)

Register / Shift register รีจิสเตอร์

- **Register n บิต:** D-FF n ตัวที่ใช้ clock ร่วมกัน (เก็บข้อมูล n บิต)
- **Shift register:** D-FF ต่อกันเป็นลูกโซ่ เลื่อนข้อมูลไปทีละบิตทุก clock (แปลง serial ↔ parallel)

Counter ตัวนับ

- **คืออะไร:** sequential circuit ที่นับเลขตามลำดับ โดย modulo-N ต้องใช้ FF อย่างน้อย **$\lceil \log_2 N \rceil$** ตัว
- **Ripple (asynchronous):** output ของ FF ตัวก่อนเป็น clock ของตัวถัดไป ทำให้ช้าและมี glitch
- **Synchronous:** ทุก FF ใช้ clock เดียวกัน
- **สัมพันธ์กับ: PC (Program Counter)** ก็เป็น register ที่เพิ่มค่าทีละ 4

FSM *Finite State Machine* เครื่องสถานะจำกัด

- **คืออะไร:** แบบจำลองของ sequential circuit ที่มี state จำนวนจำกัด และเปลี่ยน state ตาม input ทุก clock
- **ส่วนประกอบ:** state register (FF) + **next-state logic** + **output logic**
- **ขั้นตอนออกแบบ:**
  1. วาด state diagram
  2. ทำ state table
  3. **state assignment** (กำหนดรหัสบิตให้แต่ละ state)
  4. เลือกชนิด FF
  5. หาสมการด้วย K-map
  6. วาดวงจร
- **จำนวน FF ขั้นต่ำ = $\lceil \log_2(\#states) \rceil$** เช่น 10 states → 4 FFs ส่วน **one-hot** ใช้ FF เท่ากับจำนวน state (decode ง่ายกว่า)
- **สัมพันธ์กับ: control unit ของ multi-cycle processor คือ FSM** (Fetch → Decode → Exec → Mem → WB) (Ch5)

	Moore machine	**Mealy machine**
Output ขึ้นกับ	**current state เท่านั้น**	**current state + input ปัจจุบัน**
เขียน output ไว้ที่	**ในวงกลม state**	**บนเส้น transition**
จำนวน state	มักมากกว่า	มักน้อยกว่า
การตอบสนอง	ช้ากว่า 1 clock แต่ output นิ่ง	ทันที แต่อาจ glitch ตาม input

✎ **ตัวอย่าง Sequence detector "101" (overlapping)**

- **Moore: 4 states → 2 FFs**
  - S0 = เริ่มต้น, S1 = เห็น "1", S2 = เห็น "10", S3 = เห็น "101" (output 1)
  - จาก S3: ได้ 0 ไป S2, ได้ 1 ไป S1 (ใช้ "1" ตัวท้ายซ้ำได้)
- **Mealy: 3 states** (S0, S1, S2) โดย output = 1 บนเส้น S2 →(input 1)→ S1

0.7 สรุปกับดัก Digital Logic

1. **Carry out ≠ overflow:** ต้องดูว่าโจทย์ถาม signed หรือ unsigned
2. **ช่วง 2's complement ไม่สมมาตร:** 8 bits ได้ −128...+127 (และ −(−128) ใน 8 บิตก็ยังได้ −128)
3. **ฐาน 8 ใช้กลุ่มละ 3 บิต ฐาน 16 ใช้กลุ่มละ 4 บิต** และต้องเริ่มจัดกลุ่มจาก**จุดทศนิยม**
4. **แกน K-map เรียง 00, 01, 11, 10** และ**มุมทั้งสี่ติดกัน**
5. **Don't care ไม่บังคับให้วงครอบ**
6. **JK ที่ J = K = 1 คือ toggle** สถานะต้องห้ามเป็นของ **SR**
7. **Moore output อยู่ใน state** ส่วน **Mealy output อยู่บนเส้น**
8. **MUX 2^n-to-1 มี select n เส้น** (32-to-1 มี 5 เส้น ไม่ใช่ 32)

Ch1: Introduction to Computer Architecture

1.1 ภาพรวม: วิชานี้เรียนอะไร

Computer Architecture มาจาก 2 คำ: **computer** และ **architecture** (โครงสร้างและแนวคิดการออกแบบ แบบเดียวกับสถาปัตยกรรมอาคาร) จึงเป็นการศึกษา**การออกแบบ (design) การวัด (measurement) และการประเมิน (evaluation)** ของคอมพิวเตอร์ ซึ่งมีหลายระดับของ abstraction:

ระดับ (บนลงล่าง)	ตัวอย่าง
Application	โปรแกรมของผู้ใช้
Operating System / Compiler	Linux, gcc
Instruction Set Architecture	ชุดคำสั่งเช่น x86, ARM, nanoLADA
Firmware	microcode
Instruction Set Processor / I/O system	CPU และระบบ I/O
Datapath & Control	ALU, register, MUX, control unit
Digital Design	gate, flip-flop
Circuit Design	transistor
Layout	วางวงจรบน silicon

สไลด์บอกว่าเอกสารที่อธิบาย architecture หนึ่ง ๆ ควรมีอย่างน้อย **ISA, Computer Organization และ ABI**

Abstraction การซ่อนรายละเอียด

- **คืออะไร:** การแบ่งระบบเป็นชั้น โดยแต่ละชั้นเห็นเฉพาะ **interface** ของชั้นที่อยู่ติดกัน ไม่ต้องรู้รายละเอียดภายใน
- **ทำหน้าที่:** ทำให้ออกแบบระบบที่ซับซ้อนได้ (programmer ไม่ต้องรู้ว่า CPU มีกี่ pipeline stage)
- **สัมพันธ์กับ: ISA** คือ abstraction ที่สำคัญที่สุดของวิชา

ISA *Instruction Set Architecture* สถาปัตยกรรมชุดคำสั่ง

- **คืออะไร: interface/สัญญาระหว่าง software กับ hardware** กำหนดว่า software สั่งให้ hardware ทำอะไรได้บ้าง
- **กำหนดอะไรบ้าง:** instructions (operators), operands, registers และ flags, การอ้างถึง memory (addressing), instruction format (การ encode เป็น binary) และ interrupt/exception handling
- **มองได้ 2 ทาง:**
  - **Abstraction of machine for software:** programmer เขียน C แล้ว compiler แปลเป็น binary โดยไม่ต้องรู้ hardware
  - **Abstraction of software for hardware: โปรแกรม 8088 รุ่นเก่ายังรันบน Core i7 ได้** (backward compatibility)
- **ตัวอย่าง:** x86, ARM, MIPS, nanoLADA (ISA ที่ใช้สอนในวิชา)
- **สัมพันธ์กับ:** Ch3 (ISA ทั้งบท) และกำหนดค่า IC กับ CPI ใน CPU time

Computer Organization การจัดองค์ประกอบคอมพิวเตอร์ / microarchitecture

- **คืออะไร: โครงสร้างภายในที่ทำให้ ISA ทำงานได้จริง** ซึ่ง software มองไม่เห็นโดยตรง
- **ส่วนประกอบในสไลด์: ALU, Control Unit, Register, Bus, Memory subsystem, I/O subsystem** (keyboard, mouse, display, storage)
- **ตัวอย่างสำคัญ: Core i3, i5, i7 ใช้ ISA เดียวกันแต่ organization ต่างกัน** (จำนวน core, ขนาด cache, clock/Turbo Boost, Hyper-Threading)
- **สัมพันธ์กับ:** Ch4–Ch8 ทั้งหมดเป็นเรื่อง organization และกำหนดค่า CPI กับ T_c

	ISA	Organization
ตอบคำถาม	เครื่องทำ**อะไร**ได้	เครื่องทำมัน**อย่างไร**
ใครเห็น	programmer, compiler	hardware designer
เปลี่ยนแล้วต้อง recompile ไหม	**ต้อง** (binary เดิมอาจรันไม่ได้)	**ไม่ต้อง** (binary เดิมรันได้ อาจเร็วขึ้น)

ABI *Application Binary Interface* ส่วนต่อประสานระดับไบนารี

- **คืออะไร:** ข้อตกลงระดับ binary ที่ทำให้โปรแกรมหรือ library ที่ compile แยกกันทำงานร่วมกันได้
- **กำหนดอะไรบ้าง: calling convention** (ส่ง parameter ทาง register หรือ stack, caller-save/callee-save), การคืนค่า, layout ของ stack frame, alignment ของข้อมูล และ system call
- **สัมพันธ์กับ:** subroutine และ parameter passing (Ch3) ซึ่ง ISA บอกว่ามีคำสั่ง CALL ส่วน ABI บอกว่าใช้มันอย่างไร

Register รีจิสเตอร์

- **คืออะไร:** หน่วยเก็บข้อมูลขนาดเล็กที่สุดและเร็วที่สุด อยู่ใน CPU
- **ทำหน้าที่:** เป็น **temporary storage** เก็บ operand และผลลัพธ์ชั่วคราว เพื่อไม่ต้องไป memory ทุกครั้ง
- **เปรียบเทียบในสไลด์:** บวก 2 + 3 สมัยประถม → **จำ 2 ไว้ในหัว** (register) ชู 3 นิ้ว นับ 3, 4, 5 ได้ 5
- **สัมพันธ์กับ:** ยิ่งมี register มาก compiler ยิ่งเก็บตัวแปรได้มาก ทำให้เข้าถึง memory น้อยลง (แต่ต้องใช้บิตในคำสั่งเพื่อระบุ register มากขึ้น) และเป็นชั้นบนสุดของ memory hierarchy (Ch8)

Bus บัส

- **คืออะไร:** ชุดสายสัญญาณที่ใช้ร่วมกันในการส่งข้อมูล, address และ control ระหว่างส่วนต่าง ๆ
- **สัมพันธ์กับ:** ความกว้างของ bus มีผลต่อ **endian performance** และ **miss penalty** ของ cache (block ใหญ่แต่ bus แคบจะช้า)

1.2 Design Trade-offs และ Goals

Trade-off การแลกเปลี่ยนเชิงออกแบบ

- **คืออะไร:** การได้สิ่งหนึ่งโดยต้องเสียอีกสิ่งหนึ่ง ไม่มีการออกแบบไหนดีที่สุดทุกด้าน
- **คู่หลักในสไลด์: Cost vs Performance (vs Power)** ซึ่งต้องใช้ **modeling และ measurement** ในการตัดสินใจ
- **ตัวอย่างในวิชา:** single vs multi-cycle, block size ของ cache, write-back vs write-through, dynamic vs static scheduling, การให้ R-type ใช้ 5 stage

Design goals: functional requirements (market/application driven), performance goals, cost constraints, power constraints

เกี่ยวข้องทั้ง 3 ระดับ: ISA (software interface), Organization (CPU internals, memory, buses), Hardware (logic design, packaging)

Requirements	Software trends	Hardware trends
Application area: general purpose, scientific, commercial, multimedia	ต้องการ memory มากขึ้นประมาณ **2 เท่าต่อปี** (= address เพิ่ม 1 bit ต่อปี)	IC density, size, performance เพิ่มขึ้น
OS: memory management, protection, **context switching**, **interrupts**	ใช้ **compiler** มากขึ้น (ISA ออกแบบเพื่อ compiler ไม่ใช่ programmer)	**DRAM capacity 4 เท่าทุก 3 ปี** แต่ความเร็วเพิ่มช้า
Standards: IEEE floating-point, bus/I/O (PCI, USB)	optimization, scheduling	**Disk capacity 4 เท่าทุก 3 ปี**
Time to market, cost		

Moore's Law กฎของมัวร์

- **คืออะไร: ข้อสังเกต** (ไม่ใช่กฎทางฟิสิกส์) ของ **Gordon Moore** (co-founder ของ Fairchild Semiconductor และ CEO ของ Intel)
- **ใจความ: จำนวน transistor บน IC เพิ่มเป็น 2 เท่าประมาณทุก 2 ปี (18 เดือน)**
- **อย่าสับสน:** Moore's Law **ไม่ได้บอก**ว่า clock rate หรือ performance เพิ่มเป็น 2 เท่า

MIPS (หน่วยวัด) *Millions of Instructions Per Second* ล้านคำสั่งต่อวินาที

- **คืออะไร:** จำนวนคำสั่งที่ทำได้ต่อวินาที (หน่วยล้าน)
- **ข้อควรระวัง:** สไลด์ระบุว่าเป็น "หน่วยเชิงการตลาด ไม่ใช่ performance จริง" (ดู Ch2)
- **อย่าสับสน:** กับ **MIPS ที่เป็นชื่อ ISA** (RISC processor)
- **สัมพันธ์กับ:** trend **Watts per MIPS** ดีขึ้นเรื่อย ๆ (ประหยัดพลังงานขึ้น)

Cost trends: ราคา DRAM ลดลงประมาณ **40% ต่อปี** (เพราะ commoditization) ส่วนต้นทุนของ notebook เป็นแบบนี้: processor/circuit 25%, battery 25%, monitor 20%, storage 20%, RAM 10%, case 6%, video 5%, I/O/PCB 5%, keyboard/mouse 4% และมี margin 20–30%+

1.3 ทำไมต้องเรียน และ How to design

- Architecture ของ desktop/server **converged** แล้ว แต่ต้องเรียนเพราะ (1) เป็นวิศวกรคอมพิวเตอร์ (2) ต้อง**เลือก architecture ให้เหมาะกับงาน**ได้ (3) **เขียนโปรแกรมได้ดีขึ้น**
- **โปรแกรมเดียวกันเร็วไม่เท่ากัน**ได้ แม้จะใช้ algorithm, ภาษา และเครื่องเดียวกัน เพราะ data structure, compiler หรือ optimization ต่างกัน → "**Good knowledge of architecture means better program**"
  - ตัวอย่างในสไลด์: ใช้ `union` ที่รวม `unsigned short h[2]` กับ `unsigned int x` แล้วสั่ง `c.x = a.x + b.x` บวกเลข 16 บิตสองคู่ด้วยคำสั่งเดียว (คล้าย SIMD) ผลอาจต่างตาม compiler/OS
- **How to design:** เป็น**กระบวนการวนซ้ำ (iterative)** คือ Creativity → Design → **Cost/Performance Analysis** → แยก good / mediocre / bad ideas → ทำซ้ำ โดยใช้ **measurement and evaluation**

Ch2: Performance: Measurement and Analysis

2.1 ภาพรวม

บทนี้ตอบคำถามว่า "เครื่องไหนเร็วกว่า เร็วกว่ากี่เท่า และควรปรับปรุงตรงไหน" เครื่องมือหลักมี 3 อย่าง: **Speedup, CPU Time equation** และ **Amdahl's Law**

Performance สมรรถนะ

- **คืออะไร:** ความเร็วในการทำงาน ซึ่ง**ขึ้นกับบริบท (relative)** และ**เป็นส่วนกลับของเวลา**
- **สูตร:** Performance = 1 / Execution Time → เวลาสั้น = performance สูง

Speedup อัตราเร่ง / เร็วขึ้นกี่เท่า

- **คืออะไร:** อัตราส่วนบอกว่าเร็วขึ้นกี่เท่า
- **สูตร: Speedup = T_{old} / T_{new}** หรือ "X เร็วกว่า Y n เท่า" → **n = T_Y / T_X** = $Perf_X$ / $Perf_Y$
- **อ่านค่า:** n > 1 = เร็วกว่า, n < 1 = ช้ากว่า

✎ **ตัวอย่าง** A ใช้ 20 s และ B ใช้ 25 s → A เร็วกว่า B = 25 / 20 = **1.25 เท่า**

- ห้ามตอบ 20/25 = 0.8 และอย่าสับสนกับ "เร็วขึ้น 20%" (ซึ่งคิดจาก (25 – 20)/25)
- ถ้าโจทย์ถาม "กี่เท่า" ให้ใช้ **เวลาของตัวช้า ÷ เวลาของตัวเร็ว**

2.2 การวัด: Response Time vs Throughput

Response Time / Latency เวลาตอบสนอง

- **คืออะไร:** เวลาตั้งแต่เริ่มงาน**หนึ่งงาน**จนงานนั้นเสร็จ
- **มี 2 แบบ:**
  - **Elapsed time (wall-clock time):** เวลาจริงที่ผ่านไป รวมการรอ I/O และงานอื่น
  - **CPU time:** เวลาที่ CPU ทำงานให้โปรแกรมนี้จริง ไม่รวมเวลารอ
- **วัดอย่างไร:** คำสั่ง `time` ของ Unix ให้ค่า **real** (= elapsed), **user** (CPU time ใน user mode) และ **sys** (CPU time ใน kernel) ส่วน Windows ใช้ `ptime`

Throughput / Bandwidth ปริมาณงานต่อหน่วยเวลา

- **คืออะไร:** จำนวนงานที่ทำเสร็จต่อหน่วยเวลา ซึ่งเป็น **parallelism ระดับ task**
- **สัมพันธ์กับ: pipelining เพิ่ม throughput แต่ไม่ลด latency** (Ch6)

- **Computer A** ทำ 5 งานพร้อมกันเสร็จใน 10 นาที → throughput **0.5 งาน/นาที**, response time **10 นาที**
- **Computer B** ทำทีละงาน งานละ 3 นาที → throughput **0.33 งาน/นาที**, response time **3 นาที**
- → **A มี throughput ดีกว่า แต่ B มี response time ดีกว่า**

ส่งข้อมูล 20 TB:

- ผ่าน fiber 1 Gb/s ใช้ $20 \times 8 \times 10^{12} / 10^{9} = 160{,}000$ s ≈ **44 ชั่วโมง** (สไลด์ปัดเป็น 40)
- **คนแบก HDD 1 TB จำนวน 20 ลูกเดินข้ามตึกใช้ 15 นาที**
- → **คนแบกมี throughput สูงกว่ามาก** แม้ fiber จะส่งข้อมูลชิ้นแรกถึงได้เร็วกว่า

FLOPS *Floating-Point Operations Per Second* จำนวนการคำนวณทศนิยมต่อวินาที

- **คืออะไร:** หน่วยที่นิยมใช้กับงาน scientific และ GPU
- **ข้อควรระวัง:** เหมือน MIPS คือเป็นหน่วยเชิงการตลาด ไม่ได้สะท้อน performance ของงานจริงทุกงาน

ปัญหาในการวัด (Measurement Issues)

ประเด็น	ความหมาย
What to measure	จะใช้โปรแกรมอะไรวัด
Interference	เครื่องว่างหรือมีงานอื่นรันอยู่ (multi-programmed)
Reproducibility	วัดซ้ำแล้วได้ผลเหมือนเดิมไหม
Comparability	A เก่งงาน multimedia แต่แย่งานอื่น ส่วน B กลับกัน แล้วตัวไหนดีกว่า?

2.3 Benchmarks

Benchmark โปรแกรมมาตรฐานสำหรับวัดผล

- **คืออะไร:** ชุดโปรแกรมที่ใช้วัดและเปรียบเทียบ performance ซึ่งต้องเป็น**ชุดเดียวกัน (unified set)** จึงจะเทียบกันได้

ประเภท	ความหมาย	ตัวอย่าง
Real	ใช้ application จริง (มี script จำลองการใช้งานแบบ interactive/multi-user)	gcc, เกม
Microbenchmark	ใช้**ส่วนหนึ่ง**ของซอฟต์แวร์	
Kernel	วัดเฉพาะบางแง่มุมของ performance	วง loop หลักของโปรแกรม
Toy	โปรแกรมเล็ก ๆ (นิยมในอดีต)	quicksort, merge sort
Synthetic	เขียนขึ้นให้**เลียนแบบพฤติกรรม**ของโปรแกรมจริง	**Whetstone** (floating point), **Dhrystone** (integer)

SPEC *Standard Performance Evaluation Corporation* องค์กรสร้าง benchmark มาตรฐาน

- **คืออะไร:** องค์กร (SPEC.org) ที่ออก benchmark suites มาตรฐาน
- **ตัวอย่าง: SPEC CPU2017** (CPU), **SPECviewperf** (graphics), **SPECjbb2015** (Java virtual machine), **SPEC SFS2014** (file server)

2.4 การรวมผลหลายโปรแกรม: Mean

Mean	สูตร	ใช้เมื่อ
Arithmetic Mean (AM)	$(\Sigma T_i)/n$	เวลา (มีหน่วย) โดยแต่ละโปรแกรมสำคัญเท่ากัน
Weighted AM	$\Sigma w_i T_i$	เวลา โดยรู้สัดส่วน workload จริง
Harmonic Mean	$n / \Sigma(1/r_i)$	rate (เช่น MIPS)
Geometric Mean (GM)	$(\Pi r_i)^{\wedge}(1/n)$	**ratio ที่ไม่มีหน่วย (speedup, normalized time)**

✎ **ตัวอย่าง** ตัวอย่างในสไลด์ (เวลาเป็นวินาที):

	A	B	C
Program 1 (80%)	10	4	5
Program 2 (20%)	2	10	5
Total	12	14	**10**
AM	6	7	**5**
Weighted AM	0.8(10) + 0.2(2) = 8.4	0.8(4) + 0.2(10) = 5.2	**5**

- ถ้า workload เป็นตามสัดส่วนนี้ **C ดีที่สุด**
- แต่ถ้ารันแค่ Program 1 **B ดีที่สุด** → "เร็วที่สุด" ขึ้นกับ workload

✎ **ตัวอย่าง** Geometric mean ของ ratio:

	A/B	A/C	C/B
Prog 1	2.5	2	1.25
Prog 2	0.2	0.4	0.5
GM	√0.5 = **0.71**	√0.8 = **0.89**	√0.625 = **0.79**

ตรวจ: GM(A/B) ÷ GM(A/C) = 0.71 / 0.89 = 0.79 = GM(C/B) → **ผลสอดคล้องกันไม่ว่าจะใช้เครื่องไหนเป็น baseline**

- **ข้อดีของ GM + speedup: ไม่ต้องสนใจขนาด workload** (ปรับจาก 5 s → 3 s มีค่าเท่ากับปรับจาก 500 s → 300 s)
- **ข้อเสีย: GM (และ speedup) ทำนาย performance หรือเวลาจริงไม่ได้**

2.5 CPU Time Equation

CPU Time เวลาที่ CPU ใช้ทำงาน

- **คืออะไร:** เวลาที่ CPU ทำงานให้โปรแกรม (ถ้าไม่นับเวลารอ I/O)
- **สูตร: CPU Time = IC × CPI × T_c = (IC × CPI) / Clock rate**

IC *Instruction Count* จำนวนคำสั่งที่ถูก execute

- **คืออะไร:** จำนวนคำสั่ง**ที่รันจริง** (dynamic count) ไม่ใช่จำนวนบรรทัดของโค้ด เพราะ loop ทำให้คำสั่งเดิมรันหลายครั้ง
- **ขึ้นกับ: Program, ISA, Compiler**

CPI *Clock (cycles) Per Instruction* จำนวน clock cycle เฉลี่ยต่อคำสั่ง

- **คืออะไร:** แต่ละคำสั่งใช้ cycle เฉลี่ยกี่ cycle
- **ขึ้นกับ: ISA และ CPU organization** (รวมถึง cache miss และ pipeline stall)
- **ค่าทั่วไป:** RISC มักมี CPI คงที่ (เช่น 1) และ single-cycle มี CPI = 1
- **Average CPI: $CPI_{avg} = \Sigma(CPI_i \times freq_i)$** โดย $freq_i$ คือสัดส่วนของจำนวนคำสั่ง class i
- **สัมพันธ์กับ: IPC (Instructions Per Cycle) = 1/CPI** ซึ่ง superscalar ทำให้ IPC > 1 (CPI < 1)

Clock rate / Cycle time (T_c) ความถี่สัญญาณนาฬิกา / คาบเวลาหนึ่ง cycle

- **คืออะไร:** clock rate มีหน่วย MHz หรือ GHz ส่วน **T_c = 1 / clock rate** (1 GHz → 1 ns)
- **ขึ้นกับ: CPU organization (critical path) และ VLSI technology**

ตัวแปร	ขึ้นกับ	วิธีลด (Performance Improvement)
IC	Program, ISA, Compiler	redesign ISA, compiler optimization
CPI	ISA, Organization	redesign ISA, redesign organization (pipeline, cache)
T_c	Organization, VLSI technology	redesign organization, transistor ที่เร็วขึ้น

🔗 **ความสัมพันธ์** ทั้ง 3 ตัวแปร**ดึงกันไปมา**:

- RISC ลด CPI และ T_c แต่ IC เพิ่มขึ้น
- multi-cycle ลด T_c แต่ CPI เพิ่มขึ้น
- VLIW ลด IC
- pipeline ลด CPI ที่มีผล
- cache miss เพิ่ม CPI

ดังนั้น**ห้ามดูตัวแปรเดียวแล้วสรุปว่าเร็วกว่า**

✎ **ตัวอย่าง** CPU 1 GHz รัน 2000 ล้านคำสั่งใน 20 s ถ้าเพิ่ม clock เป็น 2 GHz แต่ CPI แย่ลง 1.2 เท่า จะใช้เวลาเท่าไร?

1. $CPI_{เดิม}$ = (20 s × 10^9 Hz) / (2000 × 10^6) = **10**
2. $CPI_{ใหม่}$ = 10 × 1.2 = **12**
3. $T_{ใหม่}$ = (2000 × 10^6 × 12) / (2 × 10^9) = **12 s** (เร็วขึ้น 20/12 = 1.67 เท่า ไม่ใช่ 2 เท่า)

MIPS เป็นหน่วยวัดที่ไม่ดี

```
MIPS = IC / (T × 10⁶) = Clock rate / (CPI × 10⁶)
```

✎ **ตัวอย่าง** A = **10 MIPS** @ 33 MHz และ B = **9 MIPS** @ 20 MHz โปรแกรมเดียวกันใช้ 12M คำสั่งบน A และ 10M คำสั่งบน B (เพราะ ISA ต่างกัน)

- T_A = 12M / 10M = **1.2 s** และ T_B = 10M / 9M = **1.11 s**
- → **B เร็วกว่าทั้งที่ MIPS ต่ำกว่า**

⚠ **กับดัก** **MIPS ใช้เทียบข้าม ISA ไม่ได้** เพราะ "1 คำสั่ง" ของแต่ละ ISA ทำงานได้ไม่เท่ากัน (IC ต่างกัน) และ **clock สูงกว่าก็ไม่ได้แปลว่าเร็วกว่า**

2.6 Amdahl's Law

Amdahl's Law กฎของแอมดาห์ล ("Law of diminishing returns")

- **คืออะไร:** สูตรหา **overall speedup** เมื่อเร่งได้**เพียงบางส่วน**ของงาน (โดย Gene M. Amdahl, 1922–2015)
- **สูตร: Speedup = 1 / ((1 – P) + P/n)**
  - P = **สัดส่วนของเวลาเดิม**ที่ถูกเร่ง
  - n = speedup ของส่วนนั้น
- **ขีดจำกัด:** เมื่อ n → ∞ ได้ **S_{max} = 1 / (1 – P)** → **ส่วนที่ไม่ได้เร่งเป็นตัวจำกัด**
- **หลักที่ได้: "Make the common case fast"** (เร่งส่วนที่กินเวลามากที่สุดก่อน)
- **สัมพันธ์กับ:** multicore/parallelism (Ch7) เพราะส่วน serial จำกัด speedup

✎ ตัวอย่าง **ซื้อของ:** มี 2 ส่วนคือ เดินทาง (A) ซึ่งกินเวลามากกว่า กับ เลือกซื้อ (B)

- **ทำ A เร็วขึ้น 2 เท่าได้ผลดีกว่าทำ B เร็วขึ้น 5 เท่า** เพราะ A กินเวลามากกว่า

ซื้ออาหาร: การซื้อกินเวลา 1/3 ของมื้อ ทำให้เร็วขึ้น 3 เท่า

- 1 / (2/3 + 1/9) = 9/7 ≈ **1.29**

✎ ตัวอย่าง **Exercises ท้ายบท**

1. Mail server: computation 40% เร็วขึ้น 10 เท่า (I/O 60%) → 1 / (0.6 + 0.04) = **1.5625**
2. FP เร็วขึ้น 1.6 เท่า และ FP กินเวลาครึ่งหนึ่ง → 1 / (0.5 + 0.5/1.6) = **1.23**
3. FP 25% (CPI 4.0), อื่น ๆ CPI 1.33 และลด CPI ของ FP เหลือ 2.5
    - $CPI_{เดิม}$ = 0.25(4) + 0.75(1.33) ≈ **2.0**
    - $CPI_{ใหม่}$ = 0.25(2.5) + 0.9975 = **1.6225**
    - Speedup = 2.0 / 1.6225 ≈ **1.23**

⚠ กับดัก **P ใน Amdahl ต้องเป็นสัดส่วนของ "เวลา"** ไม่ใช่ "จำนวนคำสั่ง"

- ข้อ 3 FP มี 25% ของคำสั่ง แต่กินเวลา 1.0 / 2.0 = **50%**
- ถ้าแทน P = 0.5 และ n = 4/2.5 = 1.6 จะได้ 1.23 เท่ากัน
- ถ้าแทน P = 0.25 คำตอบจะผิด

Ch3: Instruction Set Architecture

3.1 ภาพรวม

ISA คือทุกสิ่งที่ต้องรู้เพื่อเขียน assembly หรือสร้าง system software (OS, compiler, assembler) ได้แก่ **(1) instructions/operators** (RTL, CPI, format) **(2) operands, registers, flags (3) memory reference** (addressing, referencing, dereferencing) **(4) instruction format** (การ encode เป็น binary) และ **(5) interrupt & exception handler**

ประวัติของวิชา:

ยุค	เน้นเรื่อง
50s–60s	computer arithmetic
70s–80s	ISA design และ ISA for compiler (Pascal, Ada, stack frame)
90s	CPU organization, memory, ISA extension (multimedia, Nvidia/ATI)
2000s	parallelism, multi-core, 64-bit, PCIe, USB
2010s	GPU/deep learning, virtualization, distributed

IBM System/360 IBM 360 (ปี 1964)

- **คืออะไร:** สถาปัตยกรรมที่ IBM เปิดตัวในปี 1964 โดย**รวม instruction set ที่ไม่เข้ากัน 7 แบบให้เป็นแบบเดียว**
- **ความสำคัญ:** ก่อนหน้านั้นโปรแกรม**ไม่ portable** (ย้ายเครื่องแล้วต้องเขียนใหม่) และ IBM 360 เป็นจุดเริ่มของแนวคิด "ISA เดียว หลาย implementation"

ส่วนประกอบของภาษา Assembly

```
main:  ORI  $r1, $r0, #10    ; label: OPCODE dest, src, #immediate
       ORI  $r2, $r0, #20    ; $r0 = 0 เสมอ ดังนั้น $r2 = 20
       ADD  $r3, $r1, $r2    ; $r3 = $r1 + $r2
       SD   $r3, [100]       ; [100] = memory ที่ address 100  (C = 10 + 20)
```

Opcode *Operation code* รหัสคำสั่ง

- **คืออะไร:** ส่วนของคำสั่ง (binary) ที่บอกว่าเป็น**คำสั่งอะไร** เช่น ADD หรือ LW
- **สัมพันธ์กับ:** control unit **decode** opcode เพื่อสร้าง control signals ส่วน nanoLADA ใช้ opcode 6 bits

Operand / Immediate ตัวถูกดำเนินการ / ค่าคงที่ในคำสั่ง

- **Operand:** ข้อมูลที่คำสั่งใช้ อยู่ได้ใน register, memory หรือในตัวคำสั่งเอง
- **Immediate:** operand ที่**ฝังอยู่ในตัวคำสั่ง** (เขียนว่า `#10`) ไม่ต้องไปอ่านจากที่อื่น

Label ป้ายชื่อตำแหน่ง

- **คืออะไร:** ชื่อแทน address ของคำสั่ง (เช่น `main:` หรือ `Loop:`) ที่ assembler แปลงเป็นตัวเลขหรือ displacement ให้

Flag / Status register บิตสถานะ

- **คืออะไร:** บิตที่บอกผลของคำสั่งล่าสุด เช่น **Zero**, Carry, Overflow, Negative
- **สัมพันธ์กับ:** ใช้ตัดสิน branch และ **ต้องถูก save ตอนเกิด exception/subroutine** ("save status")

3.2 Design Issues และคุณสมบัติของ ISA ที่ดี

Design issues 3 ข้อ:

1. operand เก็บที่ไหน (accumulator, stack, registers, memory)
2. มี explicit operand กี่ตัว (0, 1, 2, 3)
3. ระบุ operand อย่างไร (addressing mode: immediate, direct, indirect, displacement)

ISA Matrix (คุณสมบัติที่ต้องการ):

คุณสมบัติ	ความหมาย
Compatibility	software portable รันได้ข้ามรุ่น
Generality / Completeness	รองรับ operation ได้หลากหลาย
Orthogonality	มี special case น้อย ใช้**ทุก addressing mode กับทุก data type/instruction ได้**
Regularity	ไม่ overload ความหมายของ field ในคำสั่ง
Ease of compilation	compiler สร้างโค้ดได้ง่าย
Ease of implementation	hardware สร้างได้ง่าย

3.3 ISA 5 แบบ (ตามตำแหน่งของ operand)

ทุกตัวอย่างคำนวณ **x = (a·b) + (a – (b·c))** และในอดีตนิยมแบบที่ใช้ memory เยอะ เพราะ**ตอนนั้น DRAM เร็วเมื่อเทียบกับ CPU และ transistor แพง**

Accumulator architecture สถาปัตยกรรมแบบตัวสะสม (ก่อน 1960)

- **คืออะไร:** มี register หลักตัวเดียวคือ **accumulator (ACC)** ซึ่งเป็น**ทั้ง source และ destination โดยปริยาย (implicit)** ทำให้คำสั่งระบุ address เพียง 1 ตัว (**1-address**)
- **ตัวอย่าง:** `LD A,[a]; MUL A,[b]; SD A,[x]; ...` ใช้ **11 คำสั่ง**
- **Pros:** hardware น้อยมาก ออกแบบและเข้าใจง่าย
- **Cons:** ACC เป็น **bottleneck**, ไม่เหมาะกับ parallelism/pipeline, **memory traffic สูง** (ต้องเก็บค่ากลางลง memory)

Stack architecture สถาปัตยกรรมแบบสแตก (1960s–70s)

- **คืออะไร:** operand อยู่ที่ **top of stack** โดยปริยาย คำสั่งคำนวณจึงไม่ต้องระบุ operand (**0-address**)
- **ตัวอย่าง:** `PUSH a; PUSH b; MUL; PUSH a; PUSH b; PUSH c; MUL; SUB; ADD; POP x` ใช้ **10 คำสั่ง**
- **Pros:** hardware น้อย, **code density ดี** (คำสั่งสั้น), compiler สร้างโค้ดง่าย
- **Cons:** stack เป็น bottleneck, **ข้อมูลที่ต้องการไม่ได้อยู่ top of stack เสมอ**, optimize ยาก, ไม่เหมาะกับ pipeline
- **สัมพันธ์กับ:** Java bytecode / JVM เป็น stack machine

Memory-Memory architecture สถาปัตยกรรมแบบหน่วยความจำ-หน่วยความจำ (1970s–80s)

- **คืออะไร:** คำนวณกับ operand ใน memory โดยตรง (**2–3 address**)
- **ตัวอย่าง:** แบบ 3-operand `MUL k,a,b; MUL i,b,c; SUB j,a,i; ADD x,k,j` ใช้ **4 คำสั่ง** ส่วนแบบ 2-operand ใช้ **7 คำสั่ง**
- **Pros: จำนวนคำสั่งน้อยที่สุด**, compiler ง่าย
- **Cons: memory traffic สูงมาก**, **CPI แปรผัน** (ขึ้นกับจำนวนการเข้าถึง memory) และแบบ 2-operand ต้องย้ายข้อมูลเพิ่ม
- **ตัวอย่างเครื่องจริง:** VAX

Register-Memory architecture สถาปัตยกรรมแบบรีจิสเตอร์-หน่วยความจำ (1970s–ปัจจุบัน)

- **คืออะไร:** คล้าย accumulator แต่มี**หลาย register** โดย operand หนึ่งตัวอยู่ใน memory ได้ (**2-address**)
- **ตัวอย่าง:** `LD r1,a; MUL r1,b; ...` ใช้ **10 คำสั่ง**
- **Pros:** ใช้ข้อมูลใน memory ได้เลยโดยไม่ต้อง load ก่อน, encode ง่าย, code density ดี
- **Cons: poor orthogonality** (operand สองตัวมีสิทธิ์ไม่เท่ากัน), CPI แปรผัน, อาจจำกัดจำนวน register
- **ตัวอย่างเครื่องจริง: x86**

Register-Register / Load-Store architecture สถาปัตยกรรมแบบรีจิสเตอร์-รีจิสเตอร์ (1960s–ปัจจุบัน)

- **คืออะไร:** operand ของคำสั่งคำนวณเป็น **register เท่านั้น** (**3-address**) และ **เข้าถึง memory ได้ผ่าน LOAD/STORE เท่านั้น**
- **ตัวอย่าง:** `LD r1,a; LD r2,b; LD r3,c; MUL r4,r1,r2; MUL r5,r2,r3; SUB r6,r1,r5; ADD r7,r4,r6; SD r7,x` ใช้ **8 คำสั่ง**
- **Pros: fixed-length encode ง่าย, CPI ใกล้เคียงกันทุกคำสั่ง, pipeline/parallelism ง่าย**
- **Cons:** IC สูงกว่า, performance ขึ้นกับ compiler ที่ดี
- **ตัวอย่างเครื่องจริง:** MIPS, ARM, RISC-V, **nanoLADA**

แบบ	#address	#คำสั่ง	จุดเด่น	จุดด้อย
Accumulator	1	11	HW น้อยที่สุด	bottleneck, memory traffic สูง
Stack	0	10	code density ดี	data ไม่อยู่ top เสมอ
Mem-Mem	2–3	4 / 7	คำสั่งน้อยที่สุด	memory traffic สูงที่สุด
Reg-Mem	2	10	ไม่ต้อง load ก่อน	ไม่ orthogonal

แบบ	#address	#คำสั่ง	จุดเด่น	จุดด้อย
Load-Store	3	8	**ดีที่สุดสำหรับ pipeline**	IC สูง

CISC / RISC *Complex / Reduced Instruction Set Computer*

- **CISC:** คำสั่ง**ทรงพลังและซับซ้อน** (1 คำสั่งทำงานได้มาก) มีความยาวแปรผันและ addressing mode เยอะ เช่น x86, VAX → IC ต่ำ แต่ CPI สูงและ control ซับซ้อน
- **RISC:** คำสั่งเป็น **building block พื้นฐาน** แบบ fixed-length load-store เช่น MIPS, ARM → IC สูง แต่ CPI ต่ำและ T_c สั้น
- **ใครชนะ: เมื่อมี ILP (pipeline) เข้ามา RISC ชนะ**
- **ยุค Post-RISC:** Intel (CISC) **แปลคำสั่งเป็น RISC ภายใน (micro-ops)** ก่อนทำ parallelism → เส้นแบ่งไม่ชัดแล้ว
- **ARM = Acorn RISC Machine** ซึ่งเด่นเรื่อง**ประหยัดพลังงาน** (อุปกรณ์ mobile)

3.4 Memory Organization

Memory คือ **array ขนาดใหญ่ที่ index ด้วย address** และใช้ **byte addressing** (1 address = 1 byte) ถ้าต้องการบิตเดียวให้ใช้ **bit mask** (AND/OR) ขนาดใน C: char 1, short 2, int 4, long 8, float 4, double 8, long double 16 bytes ข้อมูลหลาย byte ทำให้เกิดคำถาม 2 ข้อ คือ **ordering (endian)** และ **parallelism (alignment)**

Endianness ลำดับการเก็บไบต์

- **คืออะไร:** ข้อตกลงว่าข้อมูลหลาย byte จะเก็บ byte ไหนไว้ที่ address ต่ำ โดยอ้างถึงตัวแปรด้วย **base address = address ต่ำสุด**
- **Little endian: byte ต่ำ (LSB) อยู่ address ต่ำ** ใช้โดย **Intel**
- **Big endian: byte สูง (MSB) อยู่ address ต่ำ** ใช้โดย **IBM PowerPC, Sun** และไฟล์ **Photoshop** (เก็บแบบ big endian เสมอ)
- **Bi-endian: ARM (ตั้งแต่ ARM6)** เลือกได้ทั้งสองแบบ
- **ทำไมสำคัญ:** การส่งไฟล์ binary ระหว่าง Mac PowerPC (big) กับ Windows PC (little) ต้องแปลงลำดับ byte

เก็บ 0x1A2B3C4D	addr 00	01	02	03
Big endian	1A	2B	3C	4D
Little endian	4D	3C	2B	1A

★ จำให้ได้ **Endian กับ performance** (เมื่อ bus เล็กกว่าข้อมูล ระบบจะอ่านจาก address ต่ำไปสูง)

- **บวกเลข → little endian ดีกว่า** เพราะได้ byte ต่ำก่อน จึงบวกและส่ง carry ต่อได้ทันที
- **เปรียบเทียบมาก/น้อย → big endian ดีกว่า** เพราะได้ byte สูง (มี sign bit) ก่อน จึงตอบได้ทันที

Alignment การจัดแนวข้อมูล

- **คืออะไร:** ข้อกำหนดว่าข้อมูลขนาด s bytes ต้องเริ่มที่ address แบบไหน
- **ทำไมต้องมี:** memory อ่านได้ทีละหลาย byte พร้อมกัน (parallelism) ถ้าข้อมูลคร่อมขอบ ต้องอ่าน 2 ครั้ง
- **Restricted alignment:** ต้องมี **Base Address mod size = 0**
  - **ภาระอยู่ที่ software/compiler** (ต้องใส่ padding)
  - HW ตรวจเจอ misalignment แล้ว **trap**
  - **ไม่เสียเวลาเพิ่ม → ชนะในแง่ performance** ("make the common case fast")
- **Unrestricted alignment:** วางที่ไหนก็ได้
  - software ง่าย แต่ **HW ต้องตรวจและแก้เอง** มักต้องเข้าถึง memory **2 ครั้งขึ้นไป** (ช้า, logic แพง)
  - บางครั้งต้องมีเพื่อ backward compatibility
- **สัมพันธ์กับ: ABI** กำหนด alignment และ padding ใน struct

✎ **ตัวอย่าง** Restricted alignment ของ `char a; short b; int c; char d; long e; double f;`

Addr	0	1	2–3	4–7	8	9–15	16–23	24–31
	a	pad	b	c	d	pad 7 B	e	f

- รวม **32 bytes**
- ถ้าเรียงตัวแปรใหม่จากใหญ่ไปเล็กจะเหลือ 8 + 8 + 4 + 2 + 1 + 1 = 24 bytes

Padding ไบต์เติมช่องว่าง

- **คืออะไร:** byte ว่างที่ compiler ใส่เพื่อให้ตัวแปรถัดไป aligned
- **วิธีหาตำแหน่ง:** ตำแหน่งถัดไปต้องเป็น**พหุคูณของขนาดตัวแปร** (int → หาร 4 ลงตัว, double → หาร 8 ลงตัว)

3.5 Addressing Modes

Addressing mode วิธีระบุตำแหน่งของ operand

- **คืออะไร:** กฎที่บอกว่าจะหาค่า operand หรือ **EA (Effective Address)** ได้อย่างไร
- **ทำหน้าที่:** ทำให้โปรแกรมอ้างถึงข้อมูลแบบต่าง ๆ (ตัวแปร, array, pointer, struct) ได้สะดวก

#	Mode	ตัวอย่าง	วิธีหาค่า	ตรงกับ C
1	**Immediate**	`LDI r1, #4`	ค่าอยู่ในคำสั่ง	A = 4
2	**Register (direct)**	`ADD r1,r2,r3`	ค่าอยู่ใน register	A = B + C
3	**Displacement**	`LD r1, 100(r2)`	M[R2 + 100]	**A = G.age** (field ของ struct) หรือ local variable
4	**Register Indirect**	`LD r1, (r2)`	M[R2]	**A = *p**
5	**Index**	`LD r1, (r2+r3)`	M[R2 + R3]	**A = D[i]**

#	Mode	ตัวอย่าง	วิธีหาค่า	ตรงกับ C
6	**Direct / Absolute**	`LD r1, 1000`	M[1000]	ตัวแปร global
7	**Memory Indirect**	`LD r1, @(r2)`	M[M[R2]]	**A = **p**
8	**Auto-increment**	`LD r1, (r2)+`	M[R2] แล้ว R2 += size	**A = D[i++]**
9	**Auto-decrement**	`LD r1, (r2)-`	M[R2] แล้ว R2 −= size	**A = D[i−−]**
10	**Scale**	`LD r1, (r2)[r3*4]`	M[R2 + R3 × 4]	**F = E[i]** (int 4 bytes)

- **Register indirect แทน mode อื่นได้เกือบทั้งหมด** (คำนวณ address ใส่ register ก่อนแล้วค่อยใช้)
- **Trend ของ ISA ใหม่:** รองรับเฉพาะ **displacement, immediate และ register indirect** โดย **displacement field 12–16 bits** และ **immediate field 8–16 bits** ส่วน mode ซับซ้อนให้ **compiler แปลงเป็น mode ง่าย** และให้ HW โฟกัสที่ mode ง่าย

3.6 ประเภทคำสั่งและ Control Flow

ประเภท	หน้าที่	ตัวอย่าง
Data movement/transfer	ย้ายข้อมูลระหว่าง memory, register, I/O	LW, SW
Arithmetic	คำนวณเลข	ADD, SUB, MUL, DIV
Logic	operation ทางตรรกะ	AND, OR, XOR, NOT (Com)
Control	เปลี่ยน PC (ปกติ PC ← PC + 4)	JMP, BEQ, CALL, RET
Extended	คำสั่งเฉพาะทาง	multimedia (SIMD), encryption

PC *Program Counter* ตัวนับโปรแกรม

- **คืออะไร:** register พิเศษที่เก็บ **address ของคำสั่งถัดไป**
- **ทำงานอย่างไร:** ปกติ PC ← PC + 4 (คำสั่งยาว 4 bytes) ส่วนคำสั่ง control flow จะเขียนค่าใหม่ลง PC
- **สัมพันธ์กับ: Fetch** อ่าน MEM[PC], branch, exception (save PC) และ pipeline ที่แต่ละ stage มี PC ของคำสั่งต่างกัน

Branch vs Jump กระโดดแบบมีเงื่อนไข vs ไม่มีเงื่อนไข

- **Branch:** กระโดด**เมื่อเงื่อนไขเป็นจริง** (เช่น BEQ) มักใช้ **PC-relative displacement**
- **Jump:** กระโดด**เสมอ** (เช่น JMP) ไปยัง address ที่ระบุ
- **สัมพันธ์กับ:** เป็นต้นเหตุของ **control hazard** ใน pipeline (Ch6)

PC-relative addressing การอ้างตำแหน่งเทียบกับ PC

- **คืออะไร:** ปลายทางของการกระโดด = PC + displacement
- **ทำไมนิยม:**
  - ทำให้โปรแกรม **relocatable** (ย้ายไปวางที่ address ไหนก็ทำงานได้)
  - branch ส่วนใหญ่กระโดดใกล้ ๆ จึงใช้ field เล็กได้
- **กรณีที่ใช้ไม่ได้:** address ที่**ไม่รู้ตอน compile** เช่น **shared library / dynamic binding** ต้องใช้ **register ที่เก็บ address** (`CALL $r14`) และ symbol table

Subroutine

Subroutine / Procedure call การเรียกโปรแกรมย่อย

- **ขาเรียก:**
  1. ส่ง parameter
  2. save status
  3. **save return address**
  4. เปลี่ยน PC ไปที่ subroutine
  5. ทำงาน
  6. save return data
- **ขากลับ:** restore status แล้ว **load return address กลับเข้า PC**
- **ใครกำหนดรายละเอียด: ABI**

Parameter passing การส่งพารามิเตอร์

- **ผ่าน register:** เร็ว เช่น `LI $r3,#5; LI $r4,#4; CALL _min`
- **ผ่าน stack frame:** `PUSH #5; PUSH #4; CALL _min` แล้ว callee อ่านด้วย `LD $r3,[$r14+4]` (`$r14` = stack pointer)
- **Stack frame:** พื้นที่บน stack ของการเรียกแต่ละครั้ง (parameter, return address, local variable)

Caller-save vs Callee-save ผู้เรียกเก็บ vs ผู้ถูกเรียกเก็บ

- **Caller-save: ผู้เรียก** PUSH register ที่ตัวเองใช้ก่อน CALL แล้ว POP คืนหลังกลับมา
  - ปัญหา: caller **ไม่รู้ว่า callee ใช้ register ไหน** จึงอาจ save เกินจำเป็น
- **Callee-save: subroutine** PUSH เฉพาะ register ที่**ตัวเองจะแก้** แล้ว POP ก่อน RET → **มักคุ้มกว่า** (save เท่าที่ใช้จริง)
- **x86** มี `PUSHA/POPA` ที่ save ทุก register (**ง่ายแต่ช้า**)

3.7 Exceptions และ Interrupts

Exception ข้อยกเว้น / เหตุการณ์ที่ทำให้ control flow เปลี่ยนโดยไม่ได้เขียนไว้

- **คืออะไร: "Unprogrammed control flow"** คือการเปลี่ยน PC ที่โปรแกรมไม่ได้สั่งตรง ๆ
- **แบ่งตามแหล่ง:**
  - **Interrupt:** มาจาก **hardware ภายนอก** (เช่น keyboard, timer, disk เสร็จงาน)
  - **Trap:** มาจาก **software** (เช่น system call, หารด้วยศูนย์, misalignment, page fault)
  - Intel เรียกทั้งสองแบบว่า interrupt
- **ขั้นตอน (คล้าย subroutine):**
  1. save status
  2. **save PC**
  3. เรียก **handler**
  4. ทำงาน
  5. restore status
  6. restore PC (เช่นคำสั่ง IRET)

ISR / Handler *Interrupt Service Routine* โปรแกรมจัดการ interrupt

- **คืออะไร:** โค้ดที่รันเมื่อเกิด exception หรือ interrupt แหล่งนั้น ๆ
- **ปัญหา:** CPU ต้องรู้ว่า handler อยู่ที่ไหน ซึ่งมี 3 วิธีตามตารางด้านล่าง

วิธี	ตารางเก็บอะไร	หา handler อย่างไร	ตัวอย่าง (base 0x8000, interrupt 8)
Vector Table	**address** ของ handler	addr = **mem[Base + n × address size]** แล้วกระโดดไปที่ค่านั้น (อ่าน memory 1 ครั้ง)	32-bit address: mem[0x8000 + 8 × 4] = **mem[0x8020]**
Handler Table	**ตัวโค้ด** ของ handler (ขนาดคงที่)	addr = **Base + n × handler size** กระโดดไปได้ทันที	handler 16 B: 0x8000 + 8 × 16 = **0x8080**
Fixed Entry	ไม่มีตาราง	CPU กระโดดไป **0x1000 เสมอ** แล้ว **software อ่าน interrupt-source register** เพื่อตัดสินใจเอง	—

3.8 Instruction Format

Instruction format / Encoding รูปแบบการเข้ารหัสคำสั่ง

- **คืออะไร:** การจัด field (opcode, register, immediate) ลงใน binary ของคำสั่ง
- **มีผลต่อ: ขนาดของ code** และ **งาน decode ของ processor**
- **ขึ้นกับ:** จำนวน addressing mode และความเป็นอิสระระหว่าง opcode กับ mode
- **สิ่งที่ต้องการ (ขัดแย้งกัน):** register และ addressing mode เยอะ, คำสั่งสั้น (code เล็ก), decode/implement ง่าย

แบบ	ลักษณะ	ข้อดี	ข้อเสีย	ตัวอย่าง
Variable length	บิตแรกบอกความยาว ใช้ address specifier	รองรับ mode เยอะ **code เล็ก**	decode ยาก	x86, VAX
Fixed length	mode น้อย opcode บอก mode	**decode ง่าย ทำ pipeline ง่าย**	code ใหญ่กว่า	nanoLADA, MIPS
Hybrid	ผสมกัน			ARM Thumb (และรัน Java bytecode บนชิปได้ที่ CPI สูงขึ้น)

✎ **ตัวอย่าง ถ้าต้องการ performance แบบ fixed length ชนะ**

ตัวอย่างในสไลด์: **code ของ MIPS ใหญ่ประมาณ 2 เท่าของ VAX แต่ CPI เป็น 1/6** (สมมติ clock เท่ากัน)

- $T_{MIPS} / T_{VAX} = (2 \times IC) \times (CPI/6) / (IC \times CPI) = 2/6 = 1/3$
- → **MIPS เร็วกว่าประมาณ 3 เท่า**

3.9 Compiler และ Optimization

Compiler ตัวแปลภาษา

- **คืออะไร:** โปรแกรมแปลภาษาระดับสูงเป็น machine code โดย **ISA เป็นเป้าหมาย (target) ของ compiler**
- **Goals: correctness** (สำคัญที่สุด), speed ของโค้ด, compile time และ debugging support
- **ขั้นตอน:**
  1. front-end (ภาษา → **intermediate code**)
  2. high-level optimization
  3. global optimization
  4. **code generator** (machine-dependent)

Basic block บล็อกพื้นฐาน

- **คืออะไร:** ลำดับคำสั่งที่**ไม่มี jump/branch เข้าหรือออกตรงกลาง** (เข้าที่หัว ออกที่ท้าย)
- **สัมพันธ์กับ:** local optimization ทำภายใน basic block และใน Ch7 **basic block ใหญ่ = มีโอกาสหา ILP มาก** (loop unrolling ทำให้ใหญ่ขึ้น)

ระดับ	Optimization	ความหมาย
High-level	**Procedure integration (inlining)**	แทนการเรียกฟังก์ชันด้วยตัวโค้ดของฟังก์ชัน (ประหยัด call/return)
Local	**Common subexpression elimination (CSE)**	นิพจน์ที่คำนวณซ้ำให้คำนวณครั้งเดียว
Local	**Constant propagation**	แทนตัวแปรที่รู้ค่าคงที่ด้วยค่านั้น
Local	Stack height reduction	จัดลำดับการคำนวณให้ใช้ stack น้อยลง
Global	Global CSE	CSE ข้าม basic block
Global	**Copy propagation**	แทน `y` ด้วย `x` หลังจาก `y = x`
Global	**Code motion**	**ย้ายโค้ดที่ไม่เปลี่ยนค่าใน loop (loop-invariant) ออกนอก loop**
Machine-dependent	**Strength reduction**	ใช้ operation ที่ถูกกว่าแทน เช่น **shift แทนการคูณ** (x × 8 → x << 3)
Machine-dependent	Branch offset optimization	เลือกขนาด displacement ให้เหมาะ
Machine-dependent	**Pipeline scheduling**	จัดลำดับคำสั่งเพื่อลด stall (Ch6)

- **Copy/constant propagation:** `m=60; h=60*m; d=24*h; days=seconds/d;` → `days = seconds/86400;`
- **CSE:** `a=b+c+d; y=b+c+e;` → `x=b+c; a=x+d; y=x+e;`
- **Code motion:** loop `for(x=0;x⩽100;x++){ hs=3600; ds=hs*24; h=a[x]/hs; d=a[x]/ds; }` → ย้าย `hs=3600; ds=86400;` (พร้อมทำ constant folding) **ออกนอก loop**
- **Copy propagation ของ `A=B+C; B=A+C; D=A-B;`**
  - → `B = B+C+C` ซึ่ง**งานเพิ่มขึ้น** (บวก 2 ครั้ง)
  - → `D = (B+C)-(B+C+C) = -C` ซึ่ง**งานลดลงมาก**
  - → optimization **ไม่ได้ลดงานเสมอ** compiler ต้องชั่งน้ำหนัก

★ จำให้ได้ **ข้อสังเกตเรื่อง optimization (Effectiveness):**

- โปรแกรม "**locally simple, globally complex**"
- **bug ส่วนใหญ่ของ compiler มาจาก optimization**
- **Alias (pointer) ทำให้เก็บตัวแปรไว้ใน register ไม่ได้** เพราะ compiler ไม่รู้ว่า pointer จะแก้ตัวแปรไหน

ISA แบบไหนช่วย compiler writer:

- **Regularity** (orthogonality ระหว่าง addressing mode, data type, operation)
- **Simplicity** ("**provide primitives, not solutions**")
- ทำให้ trade-off ชัดเจน
- allow compile-time binding
- **Make the common case fast**

✎ **ตัวอย่าง Clear register แบบไหนเร็วที่สุด** (`CLR R1` , `MOV R1,0` , `SUB R1,R1,R1` , `XOR R1,R1,R1`)

- สไลด์บอกว่า **XOR แนะนำสำหรับ architecture ส่วนใหญ่**
- เหตุผล: ไม่ต้องมี immediate (คำสั่งสั้น) และ HW รู้จัก idiom นี้ (เป็น hardware-specific optimization)

Ch4: Single Cycle Processor (nanoLADA)

4.1 ภาพรวม

ISA บอกว่า CPU ต้องทำ**อะไร** ส่วนบทนี้ออกแบบ **datapath + control** ที่ทำงานนั้นจริง โดยเริ่มจากแบบที่ง่ายที่สุดคือ **single cycle**: **CPI = 1** (เริ่มคำสั่งใหม่ทุก cycle และทุก operation ของคำสั่งเป็น combinational logic) แต่ **cycle time ยาวมาก**

Processor / CPU *Central Processing Unit* หน่วยประมวลผลกลาง

- **คืออะไร:** ส่วนที่ execute คำสั่ง ประกอบด้วย **datapath** และ **control**
- **สัมพันธ์กับ:** การออกแบบ processor กำหนดค่า **CPI และ clock cycle time** (ส่วน IC ถูกกำหนดโดย ISA และ compiler)

Datapath เส้นทางข้อมูล

- **คืออะไร:** ส่วนที่**เก็บ ย้าย และประมวลผลข้อมูล** ได้แก่ PC, register file, ALU, adder, extender, memory, MUX และสายเชื่อม
- **เปรียบเทียบ:** เป็น "ถนนและเครื่องจักร" ที่ข้อมูลวิ่งผ่าน

Control unit หน่วยควบคุม

- **คืออะไร:** วงจรที่ **decode opcode** แล้วสร้าง **control signals** สั่งให้ datapath ทำงานตามคำสั่งนั้น
- **เปรียบเทียบ:** เป็น "ตำรวจจราจร" ที่บอกว่า MUX ตัวไหนเลือกทางไหน และใครได้เขียนข้อมูล
- **การ implement:** single cycle ใช้ **combinational logic, ROM หรือ PLA** ส่วน multi-cycle ใช้ **FSM หรือ microprogram**

RTL *Register Transfer Language / Level* ภาษาบรรยายการย้ายข้อมูลระหว่าง register

- **คืออะไร:** สัญกรณ์ที่บอกว่าแต่ละคำสั่ง**ย้ายหรือแปลงข้อมูลระหว่าง storage อย่างไร** เช่น `R[rd] ← R[rs] + R[rt]; PC ← PC + 4`
- **ทำหน้าที่:** เป็น "spec" ของคำสั่ง ใช้หา requirement ของ datapath
- **Logical RTL vs Physical RTL (Ch5):** logical บอกผลรวมของทั้งคำสั่ง ส่วน physical บอกว่าแต่ละ cycle ทำอะไรบน register ภายในจริง

4.2 Design Steps (5 ขั้น)

1. **Analyze ISA** หา datapath requirements: เขียนแต่ละคำสั่งเป็น **RTL** และระบุ storage elements
2. **Select components** ของ datapath และกำหนด **clocking methodology**
3. **Assemble datapath** ให้ครบ requirements โดย**เพิ่มทีละคำสั่ง**
4. **Analyze implementation** ของแต่ละคำสั่งเพื่อหาค่า **control points**
5. **Assemble control logic**

(สไลด์ชวนให้เทียบกับการออกแบบ software: requirement → design → implement)

4.3 nanoLADA ISA

nanoLADA ISA ขนาดเล็กสำหรับใช้สอนในวิชา

- **คืออะไร:** RISC แบบ **load-store, fixed-length 32 bits** มี 7 คำสั่ง และมี **3 formats**
- **ทำไมง่าย:** คำสั่งน้อย, fixed length และ **immediate กับ displacement ใช้ field เดียวกัน**

Format	Bits [31:26]	[25:21]	[20:16]	[15:11]	[10:0]
R-type	opcode (6)	rs (5)	rt (5)	rd (5)	reserved (11)
I-type	opcode (6)	rs (5)	rt (5)	immediate (16)	
J-type	opcode (6)	address/displacement (26)			

rs / rt / rd *register source / register target / register destination*

- **rs:** register ต้นทางตัวแรก
- **rt:** register ต้นทางตัวที่สอง (R-type) หรือ**ปลายทาง** (I-type เช่น LW, ORI)
- **rd:** register ปลายทางของ R-type
- แต่ละ field มี **5 bits → อ้างได้ 32 registers**

คำสั่ง	Type	Opcode	RTL
`ORI rt, rs, imm`	I	010000	R[rt] ← R[rs] OR **zero_ext**(imm); PC ← PC + 4
`ORUI rt, rs, imm`	I	010001	R[rt] ← R[rs] OR **zero_pad**(imm); PC ← PC + 4
`ADD rd, rs, rt`	R	000001	R[rd] ← R[rs] + R[rt]; PC ← PC + 4
`LW rt, imm(rs)`	I	011000	R[rt] ← MEM[R[rs] + **sign_ext**(imm)]; PC ← PC + 4
`SW rt, imm(rs)`	I	011100	MEM[R[rs] + sign_ext(imm)] ← R[rt]; PC ← PC + 4
`BEQ rs, rt, imm`	I	100100	if R[rs] = R[rt]: PC ← PC + 4 + sign_ext(imm) × 4 else PC ← PC + 4
`JMP addr`	J	110000	PC ← addr × 4

4.4 Components ของ Datapath

Register File ชุดรีจิสเตอร์

- **คืออะไร:** กลุ่ม register ทั่วไป **32 ตัว × 32 bits** (address 5 bits)
- **Ports: อ่านได้ 2 port พร้อมกัน** (Read_A, Read_B) และ**เขียนได้ 1 port** (Write_Reg)
- **สร้างจาก: D flip-flop 32 × 32 ตัว, MUX** (เลือก register ที่จะอ่าน) และ **decoder** (เลือก register ที่จะเขียน)
- **สัมพันธ์กับ:** มี write port เดียว จึงเกิด **structural hazard** เมื่อ 2 คำสั่งต้อง WB พร้อมกันใน pipeline (Ch6)

Extender วงจรขยายบิต

- **คืออะไร:** แปลง **immediate 16 bits เป็น 32 bits** มี 3 โหมด
- **Sign extend:** เติม 16 บิตบนด้วย **sign bit** ใช้กับ **LW, SW, BEQ** (offset ติดลบได้)
- **Zero extend:** เติม 16 บิตบนด้วย 0 → 0x0000_imm ใช้กับ **ORI**
- **Zero pad:** ให้ imm ไปอยู่ **16 บิตบน** แล้วเติม 0 ข้างล่าง → imm_0000 ใช้กับ **ORUI** (ทำงานคล้าย "load upper immediate")

✎ **ตัวอย่าง** สร้างค่าคงที่ 32 บิต 0x12345678 ด้วย 2 คำสั่ง:

- `ORUI r1, r0, #0x1234` → r1 = 0x12340000
- `ORI r1, r1, #0x5678` → r1 = **0x12345678**

Component	รายละเอียด
PC	register พิเศษเก็บ address ของคำสั่ง
Adder (PC)	คำนวณ PC + 4 โดย PC + 4 = {PC[31:2] + 1, 2'b00} → **ใช้ adder 30 bits ก็พอ** (2 บิตล่างเป็น 0 เสมอเพราะ word aligned)
Memory	ข้อมูลเข้าและออก 32 bits, address 32 bits, 1 read port, 1 write port (ใช้แยก instruction memory กับ data memory)
ALU	alu_ops: `00` S ← A + B, `01` S ← A OR B, `10` S ← A – B และมี **zero flag** (S = 0)

Fetch การดึงคำสั่ง

- **คืออะไร:** ขั้นตอนอ่านคำสั่งจาก memory: **Instruction ← MEM[PC]; PC ← PC + 4**
- **หมายเหตุ:** RTL ของแต่ละคำสั่งมักไม่เขียนส่วนนี้ เพราะทุกคำสั่งทำเหมือนกัน

4.5 Control Signals

สัญญาณ	ค่า 0	ค่า 1	ควบคุมอะไร
`sel_pc`	PC ปกติ	addr × 4 (JMP)	MUX เลือก PC ถัดไป
`sel_addpc`	PC + 4	PC + 4 + imm × 4	MUX เลือกผลของ branch

สัญญาณ	ค่า 0	ค่า 1	ควบคุมอะไร
`sel_wr`	**rd**	**rt**	MUX เลือก register ปลายทาง
`sel_b`	**R[rt]**	**imm** (extended)	MUX เลือก input B ของ ALU
`sel_data`	ผล ALU (S)	**ข้อมูลจาก Memory**	MUX เลือกข้อมูลที่จะเขียนลง register
`reg_wr`	ไม่เขียน	**เขียน register**	write enable ของ register file
`mem_wr`	ไม่เขียน	**เขียน memory**	write enable ของ data memory
`alu_ops`	—	—	00 ADD, 01 OR, 10 SUB

คำสั่ง	sel_pc	sel_addpc	sel_wr	sel_b	sel_data	reg_wr	mem_wr	alu_ops
ORI	0	0	1	1	0	**1**	0	01
ORUI	0	0	1	1	0	**1**	0	01
ADD	0	0	**0**	**0**	0	**1**	0	00
LW	0	0	1	1	**1**	**1**	0	00
SW	0	0	x	1	x	0	**1**	00
BEQ	0	**= zero**	x	0	x	0	0	**10**
JMP	**1**	x	x	x	x	0	0	xx

★ **จำให้ได้ ตรรกะเบื้องหลังตาราง** (ใช้สร้างตารางเองในห้องสอบได้):

- **R-type (ADD) เขียนลง rd** ส่วน **I-type เขียนลง rt** → sel_wr
- **คำสั่งที่ใช้ immediate** (ORI, ORUI, LW, SW) → sel_b = 1
- **LW เป็นคำสั่งเดียวที่เอาข้อมูลจาก memory** เขียนลง register → sel_data = 1
- **SW เป็นคำสั่งเดียวที่เขียน memory** และ **SW/BEQ/JMP ไม่เขียน register** → reg_wr = 0 ทำให้ sel_wr กับ sel_data เป็น **don't care (x)**
- **reg_wr และ mem_wr ห้ามเป็น x** เพราะจะเขียนทับข้อมูลผิด
- **BEQ ใช้ SUB** แล้วดู zero flag โดย sel_addpc = zero

Control implementation วิธีสร้างวงจรควบคุม

- **Combinational logic:** สร้างสมการของแต่ละสัญญาณโดยตรง
- **ROM:** opcode เป็น address และ control signals เป็นข้อมูล
- **PLA:** G1–G7 = AND term ของ opcode แต่ละตัว (G1 = ORI, G2 = ORUI, G3 = ADD, G4 = LW, G5 = SW, G6 = BEQ, G7 = JMP) แล้ว OR รวม:
  - `sel_pc = G7`
  - `sel_addpc = G6·zero`
  - `sel_wr = G1 + G2 + G4`
  - `sel_b = G1 + G2 + G4 + G5`
  - `sel_data = G4`
  - `reg_wr = G1 + G2 + G3 + G4`
  - `mem_wr = G5`

4.6 Critical Path

Critical path เส้นทางวิกฤต / เส้นทางที่ช้าที่สุด

- **คืออะไร:** path ของสัญญาณที่มี **delay รวมมากที่สุด** ในวงจร
- **ทำหน้าที่: กำหนด cycle time ขั้นต่ำ** (clock เร็วกว่านี้ไม่ได้ เพราะผลยังคำนวณไม่เสร็จ)
- **ใน single cycle: คำสั่งที่ยาวที่สุดคือ LW** เพราะใช้ทุกส่วน: fetch → decode → register → ALU → data memory → write back

ส่วน	(1) PC	Inst. MEM	(2) Control	(3) Register	(4) MUX	(5) ALU	(6) Data MEM	(7) MUX	รวม
ns	10	40	40	20	10	60	40	10	**230**

- **CPU time ของ 2 ล้านคำสั่ง** = $2 \times 10^6 \times 1 \times 230$ ns = **0.46 s**
- **Clock สูงสุด** ≈ 1 / 230 ns ≈ **4.35 MHz**

🔗 **ความสัมพันธ์** Single cycle ทำให้ **ทุกคำสั่งต้องใช้เวลา 230 ns เท่ากับ LW** แม้ ADD จะไม่ใช้ data memory และ JMP ใช้แค่ fetch → **เสียเวลาเปล่า** นี่คือแรงจูงใจของ **multi-cycle (Ch5)** และ **pipeline (Ch6)**

✎ **ตัวอย่าง เพิ่มคำสั่ง scale** `LD r1, (r2)[r3*4]` (Exercise)

- ต้องเพิ่ม **shifter (<< 2)** ที่ R[rt] แล้วส่งเข้า ALU ไปบวกกับ R[rs]
- ใช้ผลเป็น address ของ data memory แล้วเขียนกลับ rd
- → เพิ่ม MUX และ control signal ใหม่ และ **critical path อาจยาวขึ้น** (มี shifter อยู่ใน path)

Ch5: Multiple Cycle Processor

5.1 ภาพรวม

Multi-cycle processor โปรเซสเซอร์แบบหลายรอบสัญญาณนาฬิกา

- **คืออะไร:** แบ่งการทำงานของคำสั่งเป็น**หลายขั้นตอนเล็ก ๆ** แต่ละขั้นใช้ **1 cycle สั้น ๆ** และคำสั่งต่างชนิดใช้จำนวน cycle ต่างกัน
- **ข้อดี:**
  - **cycle time สั้นลง**
  - **คำสั่งที่ง่ายกว่าข้ามขั้นที่ไม่ใช้ได้** (BEQ ใช้ 3 cycles, LW ใช้ 5 cycles)
  - **ใช้ hardware ร่วมกันได้** (ALU ตัวเดียว, memory ตัวเดียว) เพราะใช้คนละ cycle
- **ข้อเสีย:** CPI > 1 และ control ซับซ้อนกว่า (ต้องเป็น FSM)

ปัญหาของ single cycle: cycle time ยาว เพราะ**ทุกคำสั่งต้องรอนานเท่าคำสั่งที่ช้าที่สุด**และติดอยู่กับ critical path นอกจากนี้ memory จริงก็ไม่ ideal (ใช้หลาย cycle)

Physical rule ของ memory: "fast memories are small, large memories are slow" (ในบทนี้สมมติว่ามี cache ทำให้ memory = 1 cycle ซึ่งจะกลับมาดูใน Ch8)

5.2 วิธีลด Cycle Time

Datapath partitioning การแบ่งเส้นทางข้อมูล

- **คืออะไร:** แบ่ง combinational dependency graph เป็นส่วน ๆ แล้ว**ใส่ intermediate register (latch) คั่น**
- **ทำหน้าที่:** ใช้ **หลาย cycle ที่เร็ว** แทน cycle เดียวที่ช้า
- **หลักการ: แต่ละส่วนควรใช้เวลาใกล้เคียงกัน (balanced)** เพราะ cycle time = เวลาของส่วนที่ยาวที่สุด

Intermediate registers รีจิสเตอร์คั่นกลาง (IR, REG_A, REG_B, REG_S, REG_M, REG_WR)

- **IR (Instruction Register):** เก็บคำสั่งที่ fetch มา
- **REG_A / REG_B:** เก็บ operand ที่อ่านจาก register file หรือ extender
- **REG_S:** เก็บผลจาก ALU
- **REG_M:** เก็บข้อมูลที่จะ store (SW)
- **REG_WR:** เก็บข้อมูลที่จะเขียนกลับ register
- **ทำหน้าที่: พักค่าระหว่าง cycle** เพื่อให้ cycle ถัดไปใช้ต่อได้

5 ขั้นตอนของ multi-cycle:

ขั้น	ชื่อ	ทำอะไร
1	**Fetch**	IR ← MEM[PC]; PC ← PC + 4
2	**Decode**	A ← R[rs]; B ← R[rt] หรือ ext(imm)
3	**Execute**	S ← A op B

ขั้น	ชื่อ	ทำอะไร
4	**Memory**	Data ← MEM[S] หรือ MEM[S] ← M
5	**Write back**	R[rt/rd] ← Data หรือ S

5.3 Physical RTL และจำนวน Cycle

คำสั่ง	Cycles	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5
ORI / ORUI	**4**	Fetch	A ← R[rs], B ← ext(imm)	S ← A OR B	R[rt] ← S	—
ADD	**4**	Fetch	A ← R[rs], B ← R[rt]	S ← A + B	R[rd] ← S	—
LW	**5**	Fetch	A ← R[rs], B ← sign_ext	S ← A + B	WR ← MEM[S]	R[rt] ← WR
SW	**4**	Fetch	A, B ← sign_ext, **M ← R[rt]**	S ← A + B	MEM[S] ← M	—
BEQ	**3**	Fetch	decode	if equal: PC ← PC + sign_ext(imm) × 4	—	—
JMP	**3**	Fetch	decode	PC ← addr × 4	—	—

> ★ **จำให้ได้ LW 5 · ORI/ORUI/ADD/SW 4 · BEQ/JMP 3**
> BEQ บวกแค่ imm × 4 (ไม่บวก 4 อีก) เพราะ **PC ถูกบวก 4 ไปแล้วตอน fetch**

Design steps ของ multi-cycle: ISA analysis → Components → Assemble datapath → **Datapath + Logical RTs** → **Physical RTs** → Physical RTs → Control

ปรับ datapath ได้อีก: ใช้ ALU คำนวณ PC + 4 แทน adder แยก และ**รวม instruction memory กับ data memory เป็นตัวเดียว** (เพราะใช้คนละ cycle จึงไม่ชนกัน)

5.4 Control แบบ FSM และ Microprogram

FSM control *Finite State Machine* หน่วยควบคุมแบบเครื่องสถานะ

- **คืออะไร:** control ของ multi-cycle คือ **FSM** โดยแต่ละ state = 1 cycle และกำหนด control signals ของ cycle นั้น
- **States ในสไลด์: Fetch → Decode →** แตกไปตามคำสั่ง
  - ALU_Exec → WB (ORI, ORUI, ADD)
  - ADD_Calc → MEM → (WB) (LW/SW)
  - BEQ
  - JMP
  - แล้วกลับไป **Fetch**
- **Wait state:** ถ้า memory ไม่ ideal ต้องเพิ่ม state "wait for MEM"
- **Implement ได้ 2 แบบ: sequential circuit** (state assignment + next-state truth table) หรือ **microprogram**

Microprogram / Microcode ไมโครโปรแกรม

- **คืออะไร:** implement control ด้วยการเก็บ control signals เป็น "**microinstructions**" ไว้ใน ROM แล้วมี **microsequencer** ไล่อ่านทีละตัว
- **ทำงานอย่างไร: 1 macroinstruction (คำสั่ง assembly) → หลาย microinstructions** โดยแต่ละ microinstruction ระบุ control signals ของ 1 cycle และ **Map ROM** แปลง opcode เป็นจุดเริ่มต้นของ microcode
- **ข้อดี: เปลี่ยน microprogram (Map ROM) ได้ = ทำ ISA อื่นบน hardware เดิมได้**, ออกแบบ control ซับซ้อน (CISC) ได้ง่าย
- **สัมพันธ์กับ:** Intel แปลคำสั่ง CISC เป็น micro-ops (Post-RISC, Ch3)

Macroinstruction vs Microinstruction คำสั่งระดับ ISA vs คำสั่งควบคุมภายใน

- **Macroinstruction:** คำสั่งที่ programmer เห็น (เป็นส่วนของ ISA)
- **Microinstruction:** คำสั่งภายใน CPU ระดับ control signals (เป็นส่วนของ organization)
- **ทำไมไม่เขียนโปรแกรมด้วย microinstruction โดยตรง (Exercise):**
  - microinstruction **ผูกกับ organization** เปลี่ยนรุ่น CPU แล้วโปรแกรม**ไม่ compatible**
  - microcode กว้างและยาวมาก code density แย่
  - **ISA เป็น abstraction ที่ทำให้ software portable**

★ จำให้ได้ Summary ของ Ch5:

- single cycle มีข้อเสียคือ **cycle time ยาว**
- multi-cycle **แบ่งคำสั่งเป็นขั้นตอนเล็ก ๆ และทำ 1 ขั้นต่อ cycle**
- **แบ่ง datapath ให้แต่ละส่วนเท่า ๆ กัน** เพื่อลด cycle time
- **control คือ FSM**
- **CISC ทำให้ control ซับซ้อนกว่า**
- ยังไม่ได้รวม exception ไว้ใน design

5.5 Exercises Ch5

✎ **ตัวอย่าง Potential speedup:** single cycle 210 ns, multi-cycle 45 ns ต้องได้ performance อย่างน้อยเท่าเดิม

- CPI × 45 ≤ 210 → **CPI ≤ 4.67**

✎ **ตัวอย่าง นับ cycle (cycle time 40 ns):**

- `LW` (5) + `LW` (5) + `BEQ` not taken (3) + `ADD` (4) + `SW` (4) = **21 cycles**
- 21 × 40 = **840 ns**

✎ **ตัวอย่าง Single vs multi ใครเร็วกว่า:** 100 คำสั่ง

- single 45 ns/cycle → **4500 ns**
- multi 10 ns/cycle, CPI 4.6 → 100 × 4.6 × 10 = **4600 ns**
- → **multi-cycle ช้ากว่า** (ดูข้อนี้คู่กับ pipeline ใน Ch6)

✎ **ตัวอย่าง Share ALU กับ PC adder:**

- ใส่ MUX ที่ input A ของ ALU (เลือก PC หรือ REG_A)
- ใส่ MUX ที่ input B (เลือกค่าคงที่ 4, REG_B หรือ imm × 4)
- ให้ cycle ของ Fetch ใช้ ALU คำนวณ PC + 4 ซึ่งทำได้เพราะ fetch ไม่ได้ใช้ ALU ทำอย่างอื่น

✎ **ตัวอย่าง รองรับ interrupt/exception:**

- เพิ่ม register **EPC** (Exception PC เก็บ PC ที่จะกลับไป)
- เพิ่ม register **Cause/Status**
- เพิ่ม state ใน FSM เพื่อตรวจ exception แล้วเปลี่ยน PC ไป handler (vector table / fixed entry)
- มีคำสั่ง return (IRET) ที่ restore PC และ status

- **Multi-cycle ไม่ได้เร็วกว่า single cycle เสมอ** เพราะ CPI สูงขึ้นและ register คั่นกลางมี overhead
- **CPI ของ multi-cycle ขึ้นกับ instruction mix** ต้องถ่วงด้วยความถี่ของแต่ละคำสั่ง

Ch6: Pipelining

6.1 ภาพรวม

Pipelining การประมวลผลแบบสายพาน

- **คืออะไร:** แบ่งการทำงานของคำสั่งเป็น **stages** แล้ว**ให้หลายคำสั่งทำงานซ้อนกัน**คนละ stage ในเวลาเดียวกัน ("start a new task as soon as the resources are available") เหมือนสายพานประกอบรถยนต์
- **ทำหน้าที่: เพิ่ม throughput** (ทำคำสั่งเสร็จได้ใกล้เคียง 1 คำสั่งต่อ cycle) **โดยไม่ลด latency** ของแต่ละคำสั่ง
- **สัมพันธ์กับ:** ต่อยอดจาก multi-cycle (Ch5) เพราะ resource แต่ละ stage ว่างอยู่แล้ว ("Since resources are there! Why not pipeline?") และนำไปสู่ superscalar/VLIW (Ch7)

✎ **ตัวอย่าง Laundry** (Ann, Brian, Cathy, Dave) มี 4 ขั้นตอนคือ wash, dry, fold, stash ขั้นละ 30 นาที

- **Sequential:** 4 คน × 4 ขั้น × 0.5 h = **8 ชั่วโมง**
- **Pipelined: 3.5 ชั่วโมง**
- **k loads (Pop quiz):** เวลาเติม pipe $4 \times 0.5 + (k - 1) \times 0.5$ = **$(k + 3)/2$ ชั่วโมง**

★ **จำให้ได้ Lessons learned:**

1. **Pipelining ไม่ปรับปรุง latency ของงานเดี่ยว** แต่ช่วย **throughput ของทั้ง workload**
2. หลายงานทำพร้อมกันบน **resource ต่างกัน**
3. **ความเร็วถูกจำกัดด้วย stage ที่ช้าที่สุด**
4. **Stage ยาวไม่เท่ากันทำให้ speedup ลดลง**
5. **เวลา fill (เติม) และ drain (ระบาย) pipeline ทำให้ speedup ลดลง**

Pipeline stage ขั้นตอนในสายพาน

5 stages ของ load (แบบคลาสสิก):

- **IF (Ifetch):** อ่านคำสั่งจาก instruction memory
- **ID (Reg/Dec):** decode และอ่าน register
- **EX (Exec):** ALU คำนวณ (สำหรับ load คือคำนวณ address)
- **MEM:** อ่านหรือเขียน data memory
- **WB (Wr):** เขียนผลกลับ register file

Pipeline register รีจิสเตอร์คั่นระหว่างขั้น (IF/ID, ID/EX, EX/MEM, MEM/WB)

- **คืออะไร:** register ระหว่าง stage ที่พักข้อมูลและ **control signals** ของคำสั่งที่อยู่ใน stage นั้น
- **ทำหน้าที่:** แยกคำสั่งในแต่ละ stage ไม่ให้ปนกัน
- **สัมพันธ์กับ: forwarding** ดึงข้อมูลจาก register เหล่านี้ และ register เหล่านี้มี **overhead** (setup, clk-to-Q) ทำให้ speedup ไม่ถึง k เท่าพอดี

Fill / Drain / Flush เติม / ระบาย / ล้าง pipeline

- **Fill:** ช่วงเริ่มต้นที่คำสั่งแรกยังเดินไม่ครบทุก stage (k cycles แรก)
- **Drain:** ช่วงท้ายที่ไม่มีคำสั่งใหม่เข้ามา
- **Flush:** ทิ้งคำสั่งที่ fetch มาผิด (เช่นเมื่อทาย branch ผิด)

6.2 Speedup ของ Pipeline

```
Multi-cycle (ทุกคำสั่งใช้ k cycles): T = n × k × T_c
Pipeline: T = (k + (n - 1)) × T_c (k = จำนวน stage = fill cycles)
Speedup ≈ n·k / (k + n - 1) → k เมื่อ n ใหญ่
```

✎ **ตัวอย่าง** 100 คำสั่ง:

- **Single cycle** 45 ns → **4500 ns**
- **Multi-cycle** 10 ns, CPI 4.6 → **4600 ns**
- **Ideal pipeline** 10 ns, fill 4 cycles → (4 + 99) × 10 = **1030 ns** (ถ้านับแบบ (100 + 4) × 10 จะได้ 1040 ns)
- → **pipeline เร็วที่สุด** ประมาณ **4.4 เท่า**ของ single cycle

การวาด pipeline diagram (สไลด์): แรเงาครึ่ง**ขวา** = **อ่าน** และแรเงาครึ่ง**ซ้าย** = **เขียน** (register file **เขียนครึ่งแรกของ cycle และอ่านครึ่งหลัง** จึงอ่านค่าที่เพิ่งเขียนใน cycle เดียวกันได้)

Cycle	1	2	3	4	5	6	7
I1	IF	ID	EX	MEM	WB		
I2		IF	ID	EX	MEM	WB	
I3			IF	ID	EX	MEM	WB

6.3 Hazards

Hazard (pipeline) ภาวะขัดข้องของสายพาน

- **คืออะไร:** สถานการณ์ที่ทำให้คำสั่งถัดไป**เข้า pipeline ตามจังหวะปกติไม่ได้** ("The world is not perfect")
- **มี 3 ชนิด:** structural, data และ control
- **วิธีแก้แบบง่ายที่สุดคือรอ (stall)** แต่เสีย performance → pipeline control ต้อง **detect** แล้ว **resolve** (หรือหาทางเลี่ยง)

Stall / Bubble / NOP การหยุดรอ / ฟองอากาศ / คำสั่งว่าง

- **Stall:** หยุดคำสั่งไว้ที่ stage เดิม 1 cycle ขึ้นไป
- **Bubble:** ช่องว่างใน pipeline ที่เกิดจาก stall (stage นั้นไม่ทำอะไรใน cycle นั้น)
- **NOP (No OPeration):** คำสั่งที่ไม่ทำอะไร ซึ่ง **software** แทรกเพื่อเว้นระยะแทนการให้ hardware stall

Hazard	สาเหตุ	Laundry	Processor
Structural	**resource ไม่พอ** มีคำสั่งต้องใช้พร้อมกัน	washer กับ dryer เป็นเครื่องเดียวกัน หรือ folder ไปดูทีวี	**memory เดียวใช้ทั้ง IF และ MEM, 2 คำสั่ง WB พร้อมกัน** (register file มี write port เดียว)
Data	**ต้องใช้ข้อมูลก่อนที่มันจะพร้อม** ("ข้อมูลอยู่ในอนาคต")	เสื้อกับกางเกงชุดเดียวกันซักคนละรอบ, ถุงเท้าข้างหนึ่งยังอยู่ในเครื่องซัก	`LW r2` แล้ว `ADD r4,r2,r5`
Control	**ยังไม่รู้ว่าคำสั่งถัดไปคืออะไร**	ต้องดูผลซักก่อนจึงรู้ว่าใส่ผงซักฟอกเท่าไร	**branch**

แก้ Structural Hazard

- **เพิ่ม resource** ("solve by money"): **แยก instruction cache กับ data cache** (Harvard-style) หรือซื้อเครื่องซักเพิ่ม
- **Redesign:** ถ้ามีคำสั่งยาว 3/4/5 cycles ปนกัน เช่น **LW (5) ตามด้วย ADD (4) จะ WB ใน cycle เดียวกัน**
  1. **Stall / insert bubble:** control ซับซ้อน และเสียโอกาส fetch ("no instruction is started in cycle 6")
  2. **ให้ R-type ใช้ 5 stages** (มี MEM ที่เป็น NOOP) → **ไม่มี hazard** แต่ R-type ช้าลง 1 cycle (**design trade-off**) และนี่คือเหตุผลที่ pipeline มาตรฐานให้ทุกคำสั่งผ่าน 5 stages

> ✎ **ตัวอย่าง ทำไม SW, BEQ, JMP ปนกับ R-type แล้วไม่เกิด structural hazard:**
> - **SW, BEQ และ JMP ไม่มี WB (ไม่เขียน register file)** จึงไม่แย่ง write port
> - SW ใช้ data memory เฉพาะใน stage ของตัวเอง
> - BEQ/JMP แก้ PC ตั้งแต่ stage ต้น ๆ
> - → ไม่มี resource ใดถูกใช้ซ้ำใน cycle เดียวกัน

แก้ Data Hazard

Forwarding / Bypassing การส่งข้อมูลลัด

- **คืออะไร:** ส่งผลลัพธ์จาก **pipeline register (EX/MEM, MEM/WB)** ตรงเข้า input ของ ALU **โดยไม่ต้องรอ WB**
- **ทำงานอย่างไร:** ตรวจหา**คำสั่งก่อนหน้าที่ใกล้ที่สุด**ที่จะเขียน register ที่เราต้องการ แล้วเพิ่ม **MUX** ให้เลือกข้อมูลจาก pipeline register แทน register file ("data is there, just bypass")
- **ข้อจำกัด: แก้ load-use ไม่ได้ทั้งหมด** ("ต้องมีไทม์แมชชีน") เพราะ LW ได้ข้อมูลตอน**จบ MEM** แต่คำสั่งถัดไปต้องใช้ตอน**ต้น EX ใน cycle เดียวกัน → ต้อง stall 1 cycle**
- **Data Forwarding = Data Bypassing**

Load-use hazard hazard จากการใช้ค่าที่เพิ่ง load

- **คืออะไร:** คำสั่ง LW ตามด้วยคำสั่งที่ใช้ register ที่ LW โหลด **ทันที**
- **ผล:** ถึงจะมี forwarding ก็ต้อง **stall 1 cycle** หรือ compiler ต้อง reschedule
- **ถ้าไม่มี forwarding:** ต้องรอจน WB (สไลด์: stall ADD 3 cycles หรือแทรก NOP 3 ตัว)

Delayed load / Compiler scheduling การจัดลำดับคำสั่งโดย compiler

- **คืออะไร:** ให้ **software ช่วย** โดย **ย้ายคำสั่งที่ไม่เกี่ยวข้องมาคั่น**ระหว่าง LW กับคำสั่งที่ใช้ค่า ถ้าหาไม่ได้ให้แทรก **NOP**
- **ข้อควรระวัง:** การย้ายคำสั่ง**ต้องไม่สร้าง dependency ใหม่** (RAW/WAR/WAW)

```
LW  $r2, 20($r1)          ; ไม่มี forwarding ต้องแทรก NOP 3 ตัว
NOP
NOP
NOP
AND $r4, $r2, $r5
```

แก้ Control Hazard

ใน design นี้ branch รู้ผลใน cycle ที่ 3

วิธี	ทำอย่างไร	ต้นทุน
Stall	รอจนรู้ผล branch	ช้าลง 2 cycles ทุก branch
Branch prediction	ทายว่า taken หรือ not taken แล้ว fetch ต่อไปเลย	ทายถูกไม่เสียเวลา ทายผิดเสียประมาณ 1 cycle (flush)
Branch delay slot	**นิยามใหม่ให้คำสั่งถัดจาก branch ทำงานเสมอ**	ต้องให้ compiler หาคำสั่งมาใส่

Branch prediction การทำนายทิศทาง branch

- **คืออะไร:** ทายผลของ branch ล่วงหน้าแล้วทำงานต่อไปเลย (**speculative execution**)
- **ความแม่นยำ: ทายแบบคงที่ยังถูกอย่างน้อยประมาณ 50%** ส่วน**แบบใช้ branch history ถูกประมาณ 90%**
- **ถ้าทายผิด:** ต้อง flush คำสั่งที่ทำไปตามทางผิด

Branch delay slot ช่องหน่วงหลัง branch

- **คืออะไร:** ตำแหน่งคำสั่ง**ถัดจาก branch** ที่จะ **execute เสมอไม่ว่า branch จะ taken หรือไม่** (branch มีผลช้าไป 1 คำสั่ง)
- **ทำหน้าที่:** ใช้ช่วงเวลาที่ปกติจะเสียไปให้เกิดประโยชน์
- **เงื่อนไข: ต้องมี software/compiler ช่วย** หาคำสั่งที่ไม่เกี่ยวกับเงื่อนไขของ branch มาใส่ ถ้าไม่มีให้ใส่ NOP

```
ไม่มี delay slot:              มี delay slot:
ADD $r1,$r2,$r3                 BEQ $r2,$r3,BRANCH
BEQ $r2,$r3,BRANCH              ADD $r1,$r2,$r3     ; ทำเสมอ (ย้ายมาจากข้างบน)
ORI $r4,$r5,#100                ORI $r4,$r5,#100
```

⚠ **กับดัก** ย้าย ADD เข้า delay slot ได้เพราะ **BEQ ไม่ได้ใช้** `$r1` ถ้า BEQ อ่าน `$r1` จะย้ายไม่ได้ (จะเกิด RAW)

6.4 Data Hazard 3 แบบ (Part 2)

กำหนด j = คำสั่งก่อน และ i = คำสั่งหลัง; Rregs = register ที่อ่าน และ Wregs = register ที่เขียน

RAW *Read After Write* อ่านหลังเขียน · true dependence

- **เงื่อนไข:** r ∈ **Rregs(i)** ∩ **Wregs(j)** คือ i ต้อง**อ่าน**ค่าที่ j **เขียน** แต่ถ้า i อ่านเร็วเกินไปจะได้ค่าเก่า
- **ตัวอย่าง:** `a = 7; b = a + 5;`
- **แก้: forwarding แก้ได้เกือบทุกกรณี** (ยกเว้น load-use ที่ต้อง stall 1 cycle)
- **เป็น dependency จริง** (rename ไม่ได้)

WAW *Write After Write* เขียนหลังเขียน · output dependence

- **เงื่อนไข:** r ∈ **Wregs(i)** ∩ **Wregs(j)** คือทั้งคู่เขียน register เดียวกัน แต่ถ้า i เขียนเสร็จ**ก่อน** j ค่าสุดท้ายจะผิด
- **ตัวอย่าง:** `a = data[1000]; a = 10; b = a + 3;` (load ช้าเพราะ memory)
- **แก้:** หลีกเลี่ยงด้วย design, software หรือ **out-of-order + register renaming**

WAR *Write After Read* เขียนหลังอ่าน · anti-dependence

- **เงื่อนไข:** r ∈ **Wregs(i)** ∩ **Rregs(j)** คือ i เขียนทับก่อนที่ j จะอ่านค่าเดิม
- **ตัวอย่าง:** `a = sin(x); x = 10;` (sin ต้องเตรียมนาน จึงอ่าน x ช้า)
- **ความถี่: พบได้ยากใน architecture สมัยใหม่** และหลีกเลี่ยงได้ด้วย design

- **ไม่มี RAR (Read After Read)** เพราะการอ่านไม่เปลี่ยนค่า อ่านลำดับไหนก็ได้ผลเท่ากัน
- **ใน in-order 5-stage pipeline** (อ่านใน ID และเขียนใน WB ตามลำดับเสมอ) **เกิดปัญหาจริงได้แค่ RAW**
- **WAR/WAW เป็นปัญหาเมื่อ** คำสั่งมี latency ต่างกัน หรือทำงานแบบ **out-of-order** (Ch7)
- **WAR และ WAW เป็น "name dependence"** แก้ได้ด้วย **register renaming** ส่วน RAW เป็น dependency จริง

6.5 Pipelined Datapath, Hazard Detection, Forwarding

Unified decoder ตัวถอดรหัสรวม

- **คืออะไร:** decode คำสั่ง**ครั้งเดียวที่ ID** แล้ว**ส่ง control signals ไปพร้อมคำสั่ง**ผ่าน pipeline registers แต่ละ stage จะใช้เฉพาะสัญญาณของตัวเอง
- **ตัวอย่าง "Give it a try":** ใน cycle 7 แต่ละ stage ทำคนละคำสั่ง: Fetch 100, Dcd 34, Ex 30, Mem 24, WB 20

Hazard detection unit วงจรตรวจจับ hazard

- **คืออะไร:** วงจรที่เก็บ **pending writes** (register ที่คำสั่งใน pipeline จะเขียน) แล้วเทียบกับ operand (rs, rt) ของคำสั่งที่กำลังจะ issue
- **ทำงานอย่างไร:** เมื่อคำสั่ง issue ให้ **reserve** register ปลายทาง และเมื่อเสร็จให้ลบ reservation
- **สมการ:** hazard = **((rs = rw_ex) AND regW_ex) OR ((rs = rw_mem) AND regW_mem) OR ((rs = rw_wb) AND regW_wb)** และเงื่อนไขเดียวกันกับ **rt**
- **สัมพันธ์กับ:** ใช้ **comparator** (Digital Logic) และสั่ง stall หรือเลือกทาง forwarding

- **Memory operations:** ถ้าคำสั่งเริ่ม **in-order** และ memory op อยู่ stage เดียวกันเสมอ **จะไม่มี hazard ระหว่าง memory op ด้วยกัน**
- **Load ที่มี dependency** ใช้ **"Delayed loads"** (compiler scheduling)
- **Summary ของ pipelining:**
  - ใช้ resource คนละตัวในแต่ละ step
  - เริ่มคำสั่งถัดไปขณะที่คำสั่งปัจจุบันยังไม่เสร็จ
  - **ถูกจำกัดด้วย stage ที่ยาวที่สุด (บวก fill/flush)**
  - ต้อง **detect และ resolve hazards**

6.6 Exercises Ch6

✎ **ตัวอย่าง Data hazards ในสไลด์:**

```
MOV  R1, F0        ; ใช้เวลานาน (เขียน R1 ช้า)
LI   F0, #20
LD   R2, R1, 0
ADDI R1, R1, #1
```

- **WAR บน F0:** MOV อ่าน F0 แต่ LI เขียน F0 อาจเขียนก่อน MOV อ่านเสร็จ
- **RAW บน R1:** MOV → LD และ MOV → ADDI
- **WAW บน R1:** MOV กับ ADDI (ADDI อาจเขียนก่อน MOV)
- **WAR บน R1:** LD อ่าน R1 แต่ ADDI เขียน R1

✎ **ตัวอย่าง Rescheduling** (full forwarding + delayed load + delayed branch)

```
เดิม:                          Reschedule:
Loop: LD   F0,0(R1)             Loop: LD   F0,0(R1)
      ADD  F4,F0,F2                   LD   F1,8(R1)
      SD   0(R1),F4                   ADD  F4,F0,F2
      LD   F1,8(R1)                   ADD  F5,F1,F2
      ADD  F5,F1,F2                   SD   0(R1),F4
      SD   8(R1),F5                   SUB  R1,R1,#16
      SUB  R1,R1,#16                  BNEZ R1,Loop
      BNEZ R1,Loop                    SD   -8(R1),F5   ; อยู่ใน delay slot
```

- จุดสำคัญ: **SD ที่ย้ายไปไว้หลัง SUB ต้องปรับ offset จาก 8 เป็น 8 – 16 = –8** เพราะ R1 ถูกลด 16 ไปแล้ว

Ch7: Contemporary Architecture (ILP)

7.1 ภาพรวม

Pipeline ธรรมดาทำได้ดีที่สุด CPI = 1 บทนี้หาวิธีทำให้ **CPI < 1** (หลายคำสั่งต่อ cycle) หรือลด IC โดยอาศัย **parallelism** ทั้งจาก hardware (dynamic) และ compiler (static)

Flynn's Taxonomy การจำแนกของ Flynn (Michael Flynn, 1966)

- **คืออะไร:** จำแนกคอมพิวเตอร์ตามจำนวน **instruction stream** กับ **data stream**

	Single Data	**Multiple Data**
Single Instruction	**SISD**: uniprocessor	**SIMD**: **vector processor**, GPU
Multiple Instruction	**MISD**: **fault-tolerant computing**	**MIMD**: **multicore, distributed systems**

ILP *Instruction-Level Parallelism* ความขนานระดับคำสั่ง

- **คืออะไร: จำนวนคำสั่งในโปรแกรมที่ execute พร้อมกันได้**
- **เทคนิค:** vectorization, **speculative execution** (รวม branch prediction), pipelining/superpipeline, **superscalar** (out-of-order execution, register renaming), **VLIW**
- **ข้อจำกัด:** dependency (data/control) และจำนวน resource

Speculative execution การทำงานล่วงหน้าแบบคาดเดา

- **คืออะไร:** execute คำสั่ง (หรือบางส่วน) **ก่อนที่จะรู้แน่ว่าต้องทำ** เช่นทำตามทางที่ branch prediction ทายไว้
- **ถ้าทายผิด:** ทิ้งผลลัพธ์ทั้งหมด
- **ทำหน้าที่:** เลี่ยงการ stall จาก control dependency

7.2 Vector Processor (SIMD)

Vector processor / SIMD unit *Single Instruction, Multiple Data* หน่วยประมวลผลเวกเตอร์

- **คืออะไร: ทำ operation เดียวกันกับข้อมูลหลายตัว (vector/array) ในคำสั่งเดียว**
- **ตัวอย่าง: Altivec** (Apple/Motorola/IBM), **Intel MMX, SSE2/3/4, AVX/AVX2/AVX-512**, GPU (CUDA, OpenCL)
- **ในสไลด์:** gcc `-O3` vectorize `C[i] = A[i] + B[i]` เป็น `paddd` ซึ่งบวก int 4 ตัวพร้อมกัน และ vector processor บางแบบ expand loop ภายในเอง
- **เปรียบเทียบ:** สั่ง "**บะหมี่ 10 ถ้วย**" ครั้งเดียว พ่อครัวรับ order ครั้งเดียวแล้วทำทั้งหมด

7.3 Superscalar, Superpipeline, VLIW

ตัวแปรที่ใช้: k = จำนวน stage, n = จำนวนคำสั่ง, x = degree (issue width), m = minor cycles ต่อ major cycle, T_c = major cycle

Superscalar ซูเปอร์สเกลาร์

- **คืออะไร:** มี **pipeline หลายชุด (degree x)** และ **hardware แจกคำสั่งเข้า pipeline แบบ dynamic** ทำให้ issue ได้ x คำสั่งต่อ cycle
- **ILP:** k·x (เช่น degree 2 × 5 stages = 10 คำสั่ง in flight)
- **เวลา: $T(n) = (k + (n - x)/x)\ T_c$** → speedup เทียบ pipeline ≈ **x**
- **เปรียบเทียบ:** ร้านอาหารมี**แถวเดียว** แล้ว**ผู้จัดการแยกลูกค้า**ไปแถวข้าวกับแถวบะหมี่
- **ผลข้างเคียง:** อาจเกิด **out-of-order execution** (คนสั่งบะหมี่ที่มาทีหลังได้ก่อนคนสั่งข้าว) → เสี่ยง **WAR และ WAW** แก้ด้วย **in-order completion** หรือ **register renaming**
- **ลดอะไร: ลด CPI** (CPI = 1/x)

Out-of-order execution (OoO) การทำงานไม่ตามลำดับ

- **คืออะไร:** hardware execute คำสั่งที่ **operand พร้อมแล้ว** ก่อน โดยไม่ต้องรอคำสั่งก่อนหน้าที่ยังรอข้อมูล
- **ทำหน้าที่:** ซ่อน latency และใช้ resource ให้เต็มที่ (**dynamic scheduling**)
- **ปัญหา:** WAR/WAW → ต้องใช้ register renaming และ in-order completion (commit ตามลำดับ)

Register renaming การเปลี่ยนชื่อรีจิสเตอร์

- **คืออะไร:** ให้คำสั่งที่เขียน register ชื่อเดียวกัน**ใช้ physical register คนละตัว**
- **ทำหน้าที่: กำจัด WAR และ WAW** (name dependence) ทำให้แต่ละ iteration ของ loop เป็นอิสระต่อกัน
- **ทำได้ทั้ง:** hardware (superscalar) และ compiler (ตอน loop unroll)

Superpipeline ซูเปอร์ไปป์ไลน์

- **คืออะไร:** แบ่ง major cycle เป็น **m minor cycles** แล้ว**เริ่มคำสั่งใหม่ทุก minor cycle** (pipeline ที่ลึกและเร็วขึ้น)
- **ILP:** m·k
- **เวลา: $T(n) = (k + (n - 1)/m)\ T_c$** → speedup ≈ **m**
- **Superpipelined superscalar:** ILP = m·k·x และ $T(n) = (k + (n - x)/(m \cdot x))\ T_c$

VLIW *Very Long Instruction Word* คำสั่งยาวพิเศษ

- **คืออะไร: compiler** จับหลาย operation ที่อิสระต่อกันมา**รวมไว้ใน 1 คำสั่งยาว** (x operations ต่อ word) แล้ว hardware ก็ execute ทั้ง word พร้อมกัน
- **Scheduling: static** (ทำตอน compile)
- **ILP:** k·x
- **เวลา: T(n) = (k + (n/x – x)) T_c** (ตามสไลด์) → speedup ≈ **x**
- **ลดอะไร: ลด IC** (n → n/x) โดย CPI = 1
- **เปรียบเทียบ:** ร้านอาหารมี **2 แถวแยกตั้งแต่แรก** (แถวข้าวกับแถวบะหมี่) โดยลูกค้าต้องรู้เองว่าต้องเข้าแถวไหน
- **ปัญหา: binary ไม่ compatible ข้ามรุ่น** เพราะ schedule ผูกกับ hardware (Itanium → Itanium II), Intel ประกาศ **EOL ของ Itanium ใน ม.ค. 2019**, ส่วน Transmeta Crusoe ก็ไม่ได้รับความนิยม

★ จำให้ได้ **SuperSymmetry: ที่ degree เท่ากัน superscalar, superpipeline และ VLIW ให้ performance เท่ากัน** (n ใหญ่ → speedup = x, m, x ตามลำดับ)

Architecture	IC	CPI	T_c	Potential speedup
Single cycle	n	1	L	1
Multi-cycle	n	3–5 (avg 4.2)	L/5	1.xx
Pipeline	n	1	L/5	5
Superscalar (degree k)	n	**1/k**	L/5	5k
Superpipeline (m)	n	**1/m**	L/5	5m
VLIW (x per word)	**n/x**	1	L/5	5x

✎ ตัวอย่าง **ใช้ 2 pipelines (Int + FP) พร้อมกัน**

- loop รวมใช้ 19 cycles โดย Int ใช้ 7 และ FP ใช้ 12 (fmult 7 + fadd 5)
- CPU time = max(Int, FP) → speedup = 19/12 ≈ **1.6**

7.4 Compiler & ILP

Loop unrolling การคลี่ลูป

- **คืออะไร: คัดลอกตัว loop หลายรอบไว้ใน iteration เดียว** แล้วลดจำนวนรอบลง (เช่น i –= 4)
- **ทำหน้าที่:**
  - ทำให้ **basic block ใหญ่ขึ้น** จึงมีคำสั่งอิสระมาเติม slot หรือคั่นระหว่าง load กับ use ได้
  - ลด overhead ของ loop (SUB/BNE)
- **ต้องทำร่วมกับ: register renaming** ให้แต่ละ iteration ใช้ register ต่างกัน
- **ข้อเสีย: code ใหญ่ขึ้น** และ**จำนวน register จำกัดจำนวนรอบที่ unroll ได้**

✎ ตัวอย่าง `A[i] = A[i] + C`

- **Loop ปกติ:** LW, (stall), ADD, (stall), (stall), SW, SUB, BNE = **8 cycles/iteration**
- **Unroll 4 รอบ + reschedule + renaming: 14 cycles / 4 iterations = 3.5 cycles/iter** → speedup = 8/3.5 ≈ **2.29**
- **Unroll 5 รอบ + superscalar (Int + FP): 12 cycles / 5 iter = 2.4 cycles/iter** → speedup ≈ 3.33

Software pipelining การทำ pipeline ด้วยซอฟต์แวร์

- **คืออะไร:** จัด loop ใหม่ให้แต่ละรอบ**ทำงานคนละขั้นของคนละ iteration** (เช่น SW ของ i, ADD ของ i+1, LW ของ i+2)
- **ส่วนประกอบ: start-up** code ก่อน loop และ **clean-up** code หลัง loop
- **ข้อดี: "unroll เท่าที่จำเป็น"** ได้ **performance ดีและ code ไม่ใหญ่มาก**

Dynamic vs Static scheduling จัดลำดับโดย hardware vs โดย compiler

- **Dynamic (hardware, OoO superscalar):** hardware unroll loop และ issue หลายคำสั่งแบบ dynamic
- **Static (software/compiler, VLIW):** compiler unroll และ schedule หลายคำสั่งต่อ cycle

	Dynamic	Static
ใครจัดลำดับ	hardware ตอนรัน	compiler ตอน compile
Legacy software	**เร็วขึ้นบน HW ใหม่โดยไม่ต้อง recompile**	ต้อง recompile (**poor compatibility**)
Hardware	**ซับซ้อน ใช้ power สูงกว่า**	**ง่าย ใช้ power ต่ำกว่า**
ข้อมูลที่ใช้	รู้ dependency จริงตอนรัน (เช่น cache miss)	เห็น scope กว้างกว่า แต่ **dependency บางอย่างรู้ไม่ได้ตอน compile** (pointer alias)

★ **จำให้ได้ ใครชนะ:** สไลด์สรุปว่า dynamic ทำให้ legacy software ได้ประโยชน์ ส่วน static จะดีกว่าถ้า compiler เก่งพอ แต่ **"proven to be too difficult to have good compiler" → ในทางปฏิบัติ dynamic (OoO superscalar) ชนะ**

7.5 Multicore และ Thread-Level Parallelism

Multicore ชิปหลายแกน

- **คืออะไร:** หลาย **core** (แต่ละ core เป็น superscalar) อยู่ในชิปเดียว หรือมองเป็น "multiple pipes"
- **ประเภท:** MIMD
- **สัมพันธ์กับ: Amdahl's Law** (ส่วน serial จำกัด speedup)

TLP / SMT *Thread-Level Parallelism / Simultaneous MultiThreading*

- **TLP:** ความขนานระดับ thread
- **SMT:** เมื่อ thread เดียวเติม slot ของ processor ไม่เต็ม ก็**ดึงคำสั่งจากหลาย thread** เข้า pipeline พร้อมกัน
- **Intel เรียก SMT ว่า Hyper-Threading Technology**

✎ **ตัวอย่าง Dynamic superscalar degree 2 (OoO)** จากตารางในสไลด์:

- `LW f0` issue 1, EX 2, MEM 3, WB 4
- `ADD f2,f0,f1` issue 1 แต่**รอ f0** จึง EX 5–7 และ WB 8
- `SW f2` issue 2 และ**รอ f2** จึง MEM 9
- `SUB r1` issue 2, EX 4, WB 5 → **เสร็จก่อน ADD (out-of-order)**
- `BNE` issue 3, EX 6
- iteration ถัดไปเริ่ม issue ที่ cycle 4 ซ้อนกับรอบแรก

Ch8: Memory Organization

8.1 ภาพรวม: ทำไมต้องมี Memory Hierarchy

- ยุค 70s memory **เร็วกว่า** CPU (และ 640 KB ก็พอใช้) แต่ปัจจุบัน memory **ช้ามากเมื่อเทียบกับ** CPU และต้องการอย่างน้อย 4–8 GB
- ปี 1980 CPU ยังไม่มี cache ส่วนปี 1995 มี cache 2 ระดับแล้ว

> ✎ **ตัวอย่าง** memory op ใช้ 50 cycles, op อื่นใช้ 1 cycle และมี memory op 10%
> - สัดส่วนเวลาที่ CPU รอ memory = (0.1 × 50) / (0.9 × 1 + 0.1 × 50) = 5 / 5.9 ≈ **85%**

Design goal: อยากได้ memory ที่ **large, fast และ cheap** ซึ่งทำได้ด้วย **hierarchy + parallelism** (กฎทางกายภาพคือ large memory is slow)

Memory hierarchy ลำดับชั้นของหน่วยความจำ

- **คืออะไร:** จัด memory เป็นหลายระดับ โดย**ระดับที่ใกล้ CPU จะเล็ก เร็ว และแพงต่อ bit** ส่วนระดับไกลจะใหญ่ ช้า และถูก
- **ลำดับ:** Registers → Cache (L1, L2, ...) → Main memory (DRAM) → Disk/SSD
- **ทำไมได้ผล:** อาศัย **locality** ทำให้ข้อมูลส่วนใหญ่ที่ใช้อยู่ในระดับเร็ว → **รู้สึกเหมือนมี memory ใหญ่เท่าระดับล่าง แต่เร็วเท่าระดับบน**
- **หมายเหตุ:** สไลด์เรียกระดับที่ใกล้ processor ว่า "lower level"

ระหว่าง	ใครจัดการ
Registers ↔ Memory	**Software** (compiler หรือ programmer)
Cache ↔ Memory	**Hardware** (transparent ต่อ software)
Memory ↔ Disk	**Hardware + OS** (virtual memory/paging) หรือ programmer (ผ่านไฟล์)

8.2 Memory Technology

RAM *Random Access Memory* หน่วยความจำเข้าถึงแบบสุ่ม

- **คืออะไร:** memory ที่ **"random" = เวลาเข้าถึงคงที่ทุกตำแหน่ง** (ไม่ได้แปลว่าสุ่ม)
- **ตรงข้ามกับ:**
  - **Semi-random:** disk, CD/DVD (ต้อง seek + rotate)
  - **Sequential access: tape** (เวลาแปรตามตำแหน่งแบบ linear)

DRAM *Dynamic RAM* แรมแบบไดนามิก

- **ทำจาก: capacitor** (1 transistor + 1 capacitor ต่อบิต)
- **คุณสมบัติ: high density, low power, cheap, slow** และ**ต้อง refresh เป็นระยะ** (ประจุรั่ว จึงเรียกว่า dynamic)
- **ใช้เป็น: main memory**

SRAM *Static RAM* แรมแบบสถิต

- **ทำจาก: transistor** (flip-flop ประมาณ 6 ตัวต่อบิต)
- **คุณสมบัติ: low density, high power, expensive, fast** และ**ค่าอยู่ตลอดตราบที่มีไฟ** (ไม่ต้อง refresh)
- **ใช้เป็น: cache**

8.3 Principle of Locality

Locality หลักความใกล้เคียงของการเข้าถึง

- **คืออะไร: ณ เวลาหนึ่ง โปรแกรมเข้าถึง address เพียงก้อนเล็ก ๆ** ถ้าเอาก้อนนี้ไว้ในระดับเร็ว ระบบจะรู้สึกว่าเร็วทั้งหมด
- **มี 2 แบบ:** temporal และ spatial

Temporal locality ความใกล้เคียงเชิงเวลา ("tempo" = เวลา)

- **คืออะไร: ข้อมูลที่เพิ่งใช้มีแนวโน้มถูกใช้อีกเร็ว ๆ นี้**
- **ตัวอย่าง:** ตัวแปรใน loop, เบอร์ที่เพิ่งโทรควรอยู่บนสุดของ recent list
- **Cache ใช้ประโยชน์โดย:** เก็บข้อมูลที่เพิ่งใช้ไว้ และใช้ LRU replacement

Spatial locality ความใกล้เคียงเชิงพื้นที่

- **คืออะไร: ข้อมูลที่อยู่ใกล้กับข้อมูลที่เพิ่งใช้มีแนวโน้มถูกใช้ตามมา**
- **ตัวอย่าง:** array ที่อ่านเรียงกัน, คำสั่งที่รันต่อกัน, ดึงเบอร์ของเพื่อนของเพื่อนมาพร้อมกัน, พกดินสอกับยางลบคู่กัน
- **Cache ใช้ประโยชน์โดย:** ดึงข้อมูลมาทีละ **block** (block ใหญ่ = spatial ดีขึ้น)

8.4 Cache: คำศัพท์พื้นฐาน

Cache หน่วยความจำแคช

- **คืออะไร:** หน่วยความจำ SRAM ขนาดเล็กและเร็ว อยู่ระหว่าง CPU กับ main memory โดย**เก็บสำเนาของข้อมูลที่เพิ่งใช้หรือน่าจะได้ใช้**
- **การจัดการ: hardware** ทั้งหมด (transparent ต่อ software)
- **เปรียบเทียบในสไลด์:** หอประชุม (main memory) มีนักศึกษา 100 คน (ข้อมูล) และ**ห้องอาจารย์มีเก้าอี้ 10 ตัว (cache)** เพื่อไม่ต้องเดินไปกลับหอประชุมบ่อย

คำศัพท์	ความหมาย
Hit	หาข้อมูลเจอใน cache

คำศัพท์	ความหมาย
Hit rate (hit ratio)	ความน่าจะเป็นที่หาเจอใน cache
Hit time	เวลาที่ใช้เข้าถึงข้อมูลใน cache
Miss	หาไม่เจอใน cache ต้องไปดึงจากระดับล่าง
Miss rate	**= 1 – hit rate**
Miss penalty	**เวลาที่ใช้นำข้อมูลเข้า cache** (เวลาเพิ่มเมื่อ miss)
Block (line)	ก้อนข้อมูลที่ย้ายเข้าออก cache **ทั้งก้อนเสมอ**

★ **จำให้ได้ 4 คำถามของ memory hierarchy** (ใช้ได้ทั้งกับ cache, TLB และ virtual memory):

1. **Placement:** block วางที่ไหนได้
2. **Identification:** หา block เจอได้อย่างไร (tag)
3. **Replacement:** ตอน miss จะเอา block ไหนออก
4. **Write strategy:** ตอนเขียนจะเกิดอะไรขึ้น

8.5 Direct-Mapped Cache: Tag, Index, Offset

Direct-mapped cache แคชแบบจับคู่ตรง

- **คืออะไร: แต่ละ block ของ memory วางได้ที่เดียวใน cache** (มีเก้าอี้ประจำตัว)
- **ข้อดี:** ง่าย เร็ว ใช้ comparator ตัวเดียว
- **ข้อเสีย: conflict miss** (2 block ที่ index ตรงกันจะไล่กันออก)

Index ดัชนี (บอกว่าจะนั่งเก้าอี้ตัวไหน)

- **คืออะไร:** ส่วนของ address ที่ใช้**เลือก line (หรือ set) ใน cache**
- **สูตร (ฐานสิบ): Index = (address mod cache size) div block size**
- **จำนวนบิต:** $\log_2$(จำนวน lines) โดยจำนวน lines = cache size / block size (ถ้าเป็น set-associative ให้ใช้จำนวน sets)

Tag ป้ายชื่อ (บอกว่าใครนั่งอยู่บนเก้าอี้ตัวนั้น)

- **คืออะไร:** ส่วนบนของ address ที่**เก็บไว้คู่กับข้อมูลใน cache** เพื่อตรวจว่า block ที่อยู่ใน line นั้นเป็น address ที่เราต้องการจริงไหม
- **สูตร (ฐานสิบ, direct-mapped): Tag = address div cache size**
- **การตรวจ hit: valid = 1 และ tag ที่เก็บไว้ = tag ของ address** (ใช้ comparator)

Offset (block offset + byte offset) ตำแหน่งภายใน block

- **คืออะไร:** บิตล่างสุดของ address ที่บอกว่าต้องการ **byte (หรือ word) ไหนภายใน block**
- **จำนวนบิต:** $\log_2$(block size เป็น byte)
- **การใช้งาน:** block offset เลือก word ใน block (ผ่าน MUX) และ byte offset เลือก byte ใน word

Valid bit / Dirty bit บิตบอกความถูกต้อง / บิตบอกว่าถูกแก้ไข

- **Valid bit:** บอกว่า line นี้**มีข้อมูลที่ใช้ได้หรือยัง** (ตอนเปิดเครื่อง = 0 ทั้งหมด)
- **Dirty bit:** บอกว่า block นี้**ถูกเขียนแล้วแต่ยังไม่ได้เขียนกลับ memory** ใช้กับ **write-back** (ตอน replace ถ้า dirty = 1 ต้องเขียนกลับก่อน)

```
Tag | Index | Block offset | Byte offset |
offset bits = log₂(block size B) · index bits = log₂(#sets) · tag bits = address bits - index - offset
#sets = cache size / (block size × associativity)
```

✎ **ตัวอย่าง เปรียบเทียบเก้าอี้:** cache size 10, block 1 → คนที่มี id 105 นั่ง **index 5** (105 mod 10) และติด **tag 10** (105 div 10)

✎ **ตัวอย่าง สไลด์: Tag [31:10], Index [9:2], Byte offset [1:0]**

- block = 4 bytes (1 word), 2^8 = 256 lines → **cache = 1 KB**
- **ตำแหน่งของ 0x00004098:** offset 0, **index = (0x4098 >> 2) AND 0xFF = 0x26 = 38**, **tag = 0x4098 >> 10 = 0x10 = 16**

สไลด์ (เพิ่ม spatial): Tag [31:12], Index [11:4], Block offset [3:2], Byte offset [1:0]

- block = 4 words = **16 bytes**, 256 lines → **cache = 4 KB**

8.6 Cache Performance

AMAT *Average Memory Access Time* เวลาเข้าถึงหน่วยความจำเฉลี่ย

- **สูตร: AMAT = Hit time + Miss rate × Miss penalty** (หน่วยมักเป็น cycle)
- **CPI เมื่อคิด cache: CPI = CPI_{ideal} + (memory accesses ต่อคำสั่ง × miss rate × miss penalty)** (บทก่อน ๆ สมมติว่า memory ใช้ 1 cycle)
- **Multilevel:** $AMAT_0 = Hit_0 + MR_0 \times$ **$MissPenalty_0$** โดย **$MissPenalty_0 = AMAT_1$** $= Hit_1 + MR_1 \times MissPenalty_1$
- **ลด AMAT ได้ 3 ทาง:** ลด hit time (**cache เล็ก**), ลด miss rate และลด miss penalty (**L2 cache**)

✎ **ตัวอย่าง** L1 hit 1 cycle, L1 miss 5%, L2 hit 10 cycles, L2 local miss 20% และ memory 100 cycles

- AMAT = 1 + 0.05 × (10 + 0.2 × 100) = 1 + 0.05 × 30 = **2.5 cycles**
- ถ้าไม่มี L2: 1 + 0.05 × 100 = 6 cycles

8.7 Sources of Miss (3C)

Compulsory miss / Cold miss miss ภาคบังคับ

- **คืออะไร:** miss ตอน**เข้าถึง block ครั้งแรก** (cache ยังว่าง)
- **แก้:** ทำอะไรไม่ได้มาก แต่ **prefetch** และ block ใหญ่ขึ้นช่วยได้

Capacity miss miss จากความจุไม่พอ

- **คืออะไร:** cache **เล็กเกินไปสำหรับ working set** ของโปรแกรม ทำให้ block ถูกไล่ออกก่อนจะได้ใช้อีก
- **กรณีสุดโต่ง: Ping-Pong effect** คือ cache มีแค่ 1 line ทุก address ใหม่จึง miss หมด (วางกระดาษบนโต๊ะได้ทีละแผ่น พอเปลี่ยนหน้าก็ miss)
- **แก้:** เพิ่ม cache size

Conflict miss miss จากการชนตำแหน่ง

- **คืออะไร: cache ยังมีที่ว่าง แต่ mapping บังคับให้หลาย block แย่ง index เดียวกัน**
- **แก้:** (1) เพิ่ม cache size (2) **เพิ่ม associativity** (หลาย entry ต่อ index)
- **Fully associative มี conflict miss = 0**

8.8 Cache Configurations (5 อย่าง)

1. **Cache size**
2. **Block size**
3. **Associativity**
4. **Replacement algorithm**
5. **Write management (write buffer)**

Block size ขนาดบล็อก

- **block ใหญ่ขึ้น:** spatial locality ดีขึ้น (ลด compulsory miss)
- **แต่: miss penalty สูงขึ้น** ("คนหลายคนเข้าห้องผ่านประตูเล็ก") เว้นแต่จะเพิ่มขนาด bus
- **ถ้าใหญ่เกินไป** (ที่ cache size คงที่): จำนวน block น้อยลง ทำให้ **temporal locality แย่ลงและ miss rate กลับเพิ่มขึ้น** (กราฟรูปตัว U)
- **วิธีหาค่าที่ดีที่สุด:** ต้อง benchmark (Assignment II)

Set-associative cache (n-way) แคชแบบเชื่อมโยงเป็นชุด

- **คืออะไร:** แบ่ง cache เป็น **sets** โดยแต่ละ set มี **n lines (ways)** และ block วางได้ **ที่ใดก็ได้ใน set ของมัน**
- **ผลดี: ลด conflict miss** (มีเก้าอี้ 2 ชุดในห้อง)
- **ต้นทุน:** ต้องมี **n comparators + MUX ใหญ่ → hit time อาจสูงขึ้น** และต้องมี replacement policy
- **กรณีพิเศษ: 1-way = direct-mapped** และ **n = จำนวน lines = fully associative**

Fully associative cache แคชแบบเชื่อมโยงเต็มที่

- **คืออะไร: ไม่มี index** block วางที่ไหนก็ได้
- **ผลดี: conflict miss = 0**
- **ต้นทุน:** ต้องมี **comparator เท่ากับจำนวน lines** (แพง) จึงเหมาะกับโครงสร้างเล็ก ๆ อย่าง TLB

Replacement algorithm นโยบายเลือก block ที่จะเอาออก

- **LRU (Least Recently Used):** ทิ้ง block ที่**ไม่ได้ใช้นานที่สุด** (อาศัย temporal locality)
- **Round Robin:** ผลัดกันเอาออก
- **FIFO:** ทิ้ง block ที่เข้ามาก่อน
- **Random:** สุ่มเลือก
- **Direct-mapped ไม่ต้องมี replacement algorithm** เพราะมีที่วางได้ที่เดียว จึงต้องแทนที่ตัวเดิมเสมอ
- **ในทางปฏิบัติ: แยก I-cache กับ D-cache** (code มี spatial locality สูงกว่า)

L2 cache / Multilevel cache แคชระดับที่ 2

- **คืออะไร:** cache ระดับที่สองซึ่งใหญ่กว่าแต่ช้ากว่า L1 แต่ยังเร็วกว่า memory มาก (**"เก้าอี้ที่โถงทางเดิน"**)
- **ทำหน้าที่: ลด miss penalty** ของ L1 (miss L1 อาจไป hit ที่ L2) และ**แก้ปัญหา write buffer saturation**

8.9 Write Management

Write-back เขียนกลับภายหลัง

- **คืออะไร:** เขียนลง **cache อย่างเดียว** แล้วตั้ง **dirty bit** จากนั้นจะ**เขียนลง memory ตอน block ถูก replace**
- **ข้อดี: เร็ว** และ**ดีกับการเขียนซ้ำ (repeated write)**
- **ข้อเสีย: ข้อมูล incoherent ในระบบ multiprocessor** (memory หรือ cache ของ CPU อื่นมีค่าเก่า)

Write-through เขียนผ่านทันที

- **คืออะไร: เขียนทั้ง cache และ memory ทุกครั้ง**
- **ข้อดี:** memory เป็นปัจจุบันเสมอ จึง**เหมาะกับ multiprocessor**
- **ข้อเสีย: ช้า** และ**แย่กับ repeated write** (ทุกการเขียนช้าพอ ๆ กับ miss penalty)
- **สไลด์ใช้ write-through + write allocate** เพื่อความง่าย

Write allocate vs No-write allocate ตอน write miss จะโหลด block ไหม

- **Write allocate:** ตอน write miss **โหลด block เข้า cache ก่อนแล้วเขียน** (ใช้เวลา = miss penalty + write time)
- **No-write allocate: เขียนตรงไประดับล่าง** โดยไม่โหลด block
- **นอกขอบเขต:** การเขียนแค่ส่วนหนึ่งของ block (subblock)

Write buffer บัฟเฟอร์การเขียน

- **คืออะไร: FIFO เล็ก ๆ (เช่น 4 words)** ระหว่าง cache กับ memory ใช้กับ **write-through**
- **ทำหน้าที่:** CPU เขียนลง buffer แล้ว**ทำงานต่อได้ทันที** ส่วน buffer จะค่อย ๆ ทยอยเขียนลง memory ("**มีคนวิ่งไปบอกที่หอประชุมแทน**")
- **Saturation:** ถ้า**อัตราการเขียนสูงกว่าที่ memory รับไหว** buffer จะเต็ม**ไม่ว่าจะใหญ่แค่ไหน**
- **แก้: เพิ่ม L2 cache** (วิ่งไปแค่โถงทางเดิน)

8.10 Software & Cache

- Cache **transparent** ต่อ software (ได้ประโยชน์โดยไม่ต้องแก้โค้ด) แต่ programmer ยังเขียนโค้ดให้ได้ locality ดีขึ้นได้
- **Temporal:** ใช้ตัวแปรใน scope ซ้ำต่อเนื่องกัน

Row-major vs Column-major order การเรียงอาร์เรย์ 2 มิติแบบแถวหลัก vs คอลัมน์หลัก

- **Row-major (ภาษา C):** เก็บ `data[0][0], data[0][1], …` ต่อกันทีละแถว
- **Column-major (Fortran):** เก็บทีละคอลัมน์
- **ผลต่อ cache:** ใน C **วน r ข้างนอก c ข้างใน** จะเข้าถึง memory แบบต่อเนื่อง (**spatial locality ดี**) จึงเร็วกว่าวน c ข้างนอก ซึ่งกระโดดข้ามทีละ 1 แถว

✎ **ตัวอย่าง ลด conflict miss ด้วย array of struct**

- `a[i], b[i], c[i]` จากสาม array ที่แยกกัน มักตกที่ **cache index เดียวกัน** (เมื่อขนาด array ลงตัวกับ cache) จึงไล่กันออก
- แปลงเป็น `struct {int a, b, c;} data[100]` ให้ `data[i].a/.b/.c` **อยู่ติดกันใน block เดียวกัน**

8.11 Virtual Memory (Ch8 part 2)

Virtual memory (VM) หน่วยความจำเสมือน

- **คืออะไร:** กลไก (hardware + OS) ที่ให้แต่ละโปรแกรมเห็น **virtual address space ของตัวเอง** แล้วแปลงเป็น physical address
- **ทำหน้าที่ 3 อย่าง:**
  1. **Cost:** ทำให้เหมือนมี memory มากกว่าที่มีจริงด้วยการยืม disk (**รันโปรแกรมที่ต้องใช้ 4 GB บนเครื่อง 2 GB**) ผ่าน demand loading และ swap
  2. **Relocation:** แต่ละ process มี address space ของตัวเอง (0x100 ของ A ไม่ใช่ 0x100 ของ B)
  3. **Protection:** process A **เข้าถึง memory ของ B โดยตรงไม่ได้**

Virtual address (VA) / Physical address (PA) ที่อยู่เสมือน / ที่อยู่จริง

- **VA:** address ที่ **software มองเห็น**
- **PA:** address จริงใน DRAM
- **การแปลง:** VA → PA ผ่าน **page table** (ซึ่ง TLB ช่วยให้เร็วขึ้น)

Page / Page frame หน้า (virtual) / กรอบหน้า (physical)

- **Page:** block ของ virtual address space ขนาดเท่ากัน (เช่น **4 KB**) เทียบได้กับ block ใน cache
- **Page frame:** block ของ physical memory ขนาดเท่ากับ page
- **การแบ่ง address:** VA = **VPN (Virtual Page Number)** + **page offset** และ PA = **PPN/PFN (Physical Page/Frame Number)** + **page offset (เหมือนเดิม ไม่ถูกแปล)**

Page table ตารางหน้า

- **คืออะไร:** ตารางที่ map **VPN** → **PPN** โดยแต่ละ entry (PTE) มี **valid bit**, PPN และ **access permission** (เช่น read-only)
- **ขอบเขต: มีหนึ่งตารางต่อ process** และมี **page table register** ชี้ไปที่ตารางของ process ปัจจุบัน
- **ขนาด: 2^(VPN bits) × ขนาด PTE**

Page fault ข้อผิดพลาดหน้าไม่อยู่ในหน่วยความจำ

- **คืออะไร: miss ของ virtual memory** คือ page ที่ต้องการไม่อยู่ใน physical memory (valid = 0) ต้องไปดึงจาก disk
- **ใครจัดการ: software (OS)** เพราะ hardware ไม่รู้จัก file system หรือ swap management
- **ต้นทุน: miss penalty มหาศาล** (เป็นล้าน cycles) จึงใช้ page ใหญ่ (4 KB) และ **ใช้ write-back เท่านั้น** (page out) เพราะ write-through ไปดิสก์แพงเกินไป
- **นโยบาย: demand load** (โหลดเมื่อเกิด fault) และ **replacement policy เป็นของ OS**

Swap space พื้นที่สลับบนดิสก์

- **คืออะไร:** พื้นที่บน secondary storage ที่ยืมมาเก็บ page เมื่อ physical memory ไม่พอ
- **มุมมอง:** physical memory ทำหน้าที่เป็น **"cache" ของ disk**

✎ **ตัวอย่าง สไลด์:** VA 32 bits = **VPN 20 + offset 12** และ PA = **PPN 18 + offset 12**

- **Page size** = 2^{12} = **4 KB**
- **Physical memory** = 2^{30} = **1 GB**
- **Page table** = 2^{20} × 4 B = **4 MB ต่อ process**

TLB *Translation Lookaside Buffer* บัฟเฟอร์จำผลการแปลงที่อยู่

- **คืออะไร: cache ของ page table entries ที่เพิ่งใช้**
- **ทำไมต้องมี:** ทุกการเข้าถึง memory ต้องแปล VA → PA ถ้าต้องไปอ่าน page table ใน memory ทุกครั้งจะ**ช้าลงเท่าตัว**
- **คุณสมบัติ:**
  - เข้าถึงเร็วพอ ๆ กับ cache
  - **ขนาดเล็ก (ประมาณ 128–256 entries)** จึงทำเป็น **fully associative** ได้ (เครื่องระดับกลางใช้ small n-way)
- **ผลต่อ performance: TLB miss มีผลต่อ performance อย่างมีนัยสำคัญ**
- **สัมพันธ์กับ:** cache (overlapped access) และ context switch (ต้อง flush)

8.12 Overlapped Cache & TLB Access

Overlapped (VIPT) access เข้าถึง cache กับ TLB พร้อมกัน

- **คืออะไร:** ใช้ **bits บนของ VA** เปิด TLB **พร้อมกับใช้ bits ล่าง (page offset ซึ่งไม่เปลี่ยนตอนแปล)** เป็น index ของ cache แล้วค่อยเทียบ tag ของ cache กับ PA ที่ได้จาก TLB
- **ทำหน้าที่:** ลดเวลา เพราะไม่ต้องรอแปล address เสร็จก่อนจึงเปิด cache
- **เงื่อนไข: index + offset ของ cache ต้องอยู่ในส่วน page offset** หรือก็คือ **cache size / associativity ≤ page size**

```
IF cache hit AND (cache tag = PA)                    → ส่งข้อมูลให้ CPU
ELSE IF (cache miss OR tag ≠ PA) AND TLB hit          → อ่าน memory ด้วย PA จาก TLB
ELSE                                                 → แปล VA แบบปกติ (เดิน page table)
```

✎ **ตัวอย่าง กรณีที่ทำไม่ได้:** cache 8 KB direct-mapped กับ page 4 KB

- cache ใช้ **13 bits** แต่ page offset มีแค่ **12 bits** (bit 12 เปลี่ยนตอนแปล)

ทางแก้:

1. ใช้ **page 8 KB**
2. ใช้ **2-way set associative** (เหลือ 4 KB ต่อ way)
3. ให้ **software รับประกันว่า VA[12] = PA[12]**

Exercise: page 4 KB + cache 32 KB → ต้องเป็น **8-way** (32/8 = 4 KB ต่อ way)

Virtually addressed cache แคชที่ใช้ virtual address

- **คืออะไร:** เข้า cache ด้วย VA ก่อนแปล
- **ข้อดี:** เร็ว (ไม่ต้องรอ TLB)
- **ปัญหา:**
  - **shared memory / aliasing** (VA ต่างกันจากต่าง process map ไป PA เดียวกัน)
  - ต้องจัดการตอน context switch

✎ ตัวอย่าง **Process switch ต้องทำอะไร**

- **TLB: flush** (หรือติด ASID/PID ไว้ใน entry) เพราะ mapping เป็นของ process เก่า
- **Page table: ไม่ต้องลบ** แค่เปลี่ยน **page table register** ให้ชี้ไปที่ table ของ process ใหม่
- **Cache:** ถ้าเป็น **physically addressed** ไม่ต้อง flush ถ้าเป็น **virtually addressed** ต้อง flush หรือใส่ PID tag

🔗 ความสัมพันธ์ **Cache vs Virtual Memory vs TLB** ใช้หลักเดียวกัน (4 คำถาม) แต่ต่างกันที่ขนาดและผู้จัดการ

	Cache	TLB	Virtual memory
เป็น cache ของ	main memory	**page table**	**disk**
หน่วย	block (16–64 B)	PTE	page (4 KB)
Miss เรียกว่า	cache miss	TLB miss	**page fault**
จัดการ miss โดย	hardware	hardware (หรือ OS)	**OS (software)**
Placement	direct / n-way / fully	**fully assoc.** (เล็ก)	ที่ไหนก็ได้ (fully)
Write policy	write-back หรือ write-through	—	**write-back เท่านั้น**

นอกขอบเขต: overlay, segmentation, inverted page table, multilevel page table, virtualized page table

---

Ch9: Storage Architecture

9.1 Storage และ Interface

SSD / HDD *Solid State Drive / Hard Disk Drive*

- **SSD:** เก็บข้อมูลบน **flash memory** ไม่มีชิ้นส่วนเคลื่อนไหว เร็ว
- **HDD: จานแม่เหล็กหมุน (mechanical)** ต้อง seek + rotate จึงเป็น semi-random access

Interface	ความเร็วในสไลด์	ใช้ที่
SAS (Serial Attached SCSI)	SAS-3 12 Gb/s	server
SATA (Serial ATA)	3.0: 6 Gb/s	PC ทั่วไป
PCIe (PCI Express)	3.0 x4: 31.5 Gb/s	NVMe SSD
Fibre Channel	128 Gb/s	server, SAN
USB	10 Gb/s	external
PATA (Parallel ATA)	1064 Mb/s	รุ่นเก่า
Parallel SCSI	40–2560 Mb/s	รุ่นเก่า

⚠ **กับดัก Protocol/interface ≠ storage technology** (SSD อาจต่อผ่าน SATA หรือ PCIe ก็ได้)

Storage virtualization การจำลองพื้นที่จัดเก็บ

- **คืออะไร:** "presenting a **logical view** of the physical storage" คือมองสื่อทุกชนิดเป็น **single pool** ได้ logical storage หนึ่ง (หรือหลาย) ก้อนจาก physical หลายก้อน
- **ตัวอย่าง:** RAID, JBOD, SAN

9.2 RAID

RAID *Redundant Array of Independent (เดิมคือ Inexpensive) Disks* ชุดดิสก์ที่มีความซ้ำซ้อน

- **คืออะไร: storage virtualization** ที่รวมดิสก์หลายลูกเป็นลูกเดียวในมุมมองของระบบ
- **ผู้คิดค้น: David Patterson, Garth Gibson, Randy Katz (UC Berkeley, 1987)**
- **ทำไมต้องมี:** ยุค 60s–70s ดิสก์ที่เร็วต้องแพงมาก (เช่น RAM disk ที่มีแบตเตอรี่) RAID ทำให้**ดิสก์ราคาถูก (commodity) ได้ performance เทียบดิสก์แพง**
- **แต่ละ level** มีจุดสมดุลระหว่าง **reliability, availability, performance และ capacity** ต่างกัน

คำศัพท์	ความหมาย
Striping	กระจายข้อมูลเป็นชิ้น ๆ ไปหลายดิสก์ อ่าน/เขียนขนานกัน → **เร็ว**
Mirroring	เก็บสำเนาเหมือนกันทุกบิตบนหลายดิสก์ → **ทนเสีย**

คำศัพท์	ความหมาย
Parity	ข้อมูลตรวจสอบ (XOR) ที่ใช้**กู้ข้อมูลของดิสก์ที่เสีย**ได้ 1 ลูก
Hamming code	รหัสแก้ไขข้อผิดพลาดระดับบิต (ใช้ใน RAID 2)

Level	วิธี	Min disks	ทนเสียได้	Capacity	จุดเด่น / ด้อย
RAID 0	**Stripe**	**2**	**0 ลูก**	N	**เร็วที่สุด แต่ reliability ต่ำ** (เสีย 1 ลูกข้อมูลหายหมด)
RAID 1	**Mirror**	**2**	1	N/2	ดีเมื่อ **read และ reliability สำคัญ**
RAID 2	**Bit-level + Hamming code**	—	1	—	**แทบไม่ใช้**
RAID 3	**Byte-level** parity (ดิสก์เฉพาะ)	3	1	N – 1	ดีกับ **long sequential read/write** แย่กับ random (แทบไม่ใช้)
RAID 4	**Block-level** parity บน**ดิสก์เฉพาะ**	3	1	N – 1	**read ดี write ช้า (parity disk เป็นคอขวด)**
RAID 5	Block-level parity **กระจายทุกดิสก์**	**3**	1	**N – 1**	**สมดุลระหว่าง performance กับ redundancy**
RAID 6	**2 parities**	**4**	**2**	**N – 2**	ทนเสียได้ 2 ลูก

★ จำให้ได้ **ทำไม RAID 5 เร็วกว่า RAID 4:**

- RAID 4 ต้องเขียน parity ลง**ดิสก์ลูกเดียว**ทุกครั้งที่มีการเขียน ดิสก์ลูกนั้นจึงเป็น**คอขวด**
- RAID 5 **กระจาย parity ไปทุกดิสก์** การเขียนหลาย stripe จึงทำขนานกันได้

Nested RAID RAID ซ้อน

- **RAID 01:** mirror ของ stripes
- **RAID 10:** stripe ของ mirrors
- **RAID 50:** stripe ของ RAID 5

JBOD / Span *Just a Bunch Of Disks* รวมดิสก์ต่อกันเฉย ๆ

- **คืออะไร:** ต่อดิสก์หลายลูกให้เป็นก้อนเดียว (span) โดย**รวมดิสก์ที่ขนาดต่างกันได้**
- **ข้อจำกัด: ไม่มี redundancy** (ไม่ใช่ RAID)

⚠ **กับดัก** **RAID ไม่ใช่ backup** เพราะไม่ป้องกันการลบไฟล์ผิด, ไวรัส หรือภัยที่ทำลายทุกดิสก์พร้อมกัน และ **RAID 0 ไม่มี redundancy เลย**

9.3 Network Storage

Block storage vs File storage vs Object storage การเก็บข้อมูลระดับบล็อก / ไฟล์ / ออบเจกต์

- **Block:** ให้บริการเป็น **block/sector** ดิบ ๆ โดย **client สร้าง file system เอง** (เหมือนดิสก์เปล่า)
- **File:** ให้บริการเป็น **ไฟล์และโฟลเดอร์** โดย **server จัดการ file system และ permission**
- **Object:** เก็บเป็น object (ข้อมูล + metadata + ID) เข้าถึงผ่าน API เช่น Amazon S3

SAN *Storage Area Network* เครือข่ายพื้นที่จัดเก็บ

- **คืออะไร: block storage ผ่าน network** (เช่น **iSCSI**)
- **ลักษณะ:**
  - มักต้องใช้ **hardware เฉพาะหรือ fiber**
  - **OS ของ client เห็นเป็น blocks/sectors และจัดการ file system เอง**
  - **มักให้ระบบเดียวใช้ (ไม่ share)**
- **เทคโนโลยีคล้ายกัน:** iATA, AoE (ไม่ค่อยนิยม)

NAS *Network Attached Storage* ที่เก็บข้อมูลต่อผ่านเครือข่าย

- **คืออะไร: file storage ผ่าน network** ("network drive" เช่น map drive ใน Windows)
- **ลักษณะ: server จัดการ file system และ permission** และ **share ระหว่าง clients ได้ง่าย**
- **Protocols:**
  - **NFS** (Network File System)
  - **CIFS/SMB** (Common Internet File System / Server Message Block)
  - AFP (Apple Filing Protocol, deprecated)
  - FTP, SFTP

iSCSI *Internet Small Computer System Interface*

- **คืออะไร:** ส่งคำสั่ง SCSI (block) ผ่านเครือข่าย IP ซึ่งเป็นวิธีหลักในการทำ SAN บน Ethernet

	SAN	**NAS**
ระดับ	**Block**	**File**
File system อยู่ที่	**Client**	**Server**
Sharing	ไม่นิยม share	**share ง่าย**
Protocol	**iSCSI**, Fibre Channel	**NFS, CIFS/SMB**
Cloud	**Amazon EBS**, Google Persistent Disk	Google Cloud Filestore

✎ ตัวอย่าง **ประเภท storage ของ cloud (Exercise):**

- **Amazon S3 = object**
- **Amazon EBS = block**
- **Amazon DynamoDB = NoSQL key-value database** (ไม่ใช่ block หรือ file)
- **Google Cloud Storage bucket = object**
- **Google Drive, OneDrive, Dropbox, iCloud = file** (sync/share ไฟล์ ซึ่งภายในเก็บบน object storage)

Deduplication การกำจัดข้อมูลซ้ำ

- **คืออะไร:** ระบบหลายระบบมักมีไฟล์หรือ block เหมือนกัน จึง**เก็บไว้ชุดเดียวแล้วชี้ร่วมกัน**เพื่อประหยัด physical space

Real-world considerations: performance, price, block vs file vs object, sharing, reliability, redundancy, deduplication

Formula Sheet

เรื่อง	สูตร
Performance	Performance = 1 / Execution time
Speedup ("X เร็วกว่า Y n เท่า")	$n = T_Y / T_X = T_{old} / T_{new}$
CPU time	**$IC \times CPI \times T_c = IC \times CPI / f$**
Cycle time	$T_c = 1 / f$ (1 GHz → 1 ns)
Average CPI	$\Sigma(CPI_i \times freq_i)$
MIPS	$IC / (T \times 10^6) = f / (CPI \times 10^6)$
Weighted AM	$\Sigma\ w_i T_i$
Geometric mean	$(\Pi\ r_i)^{(1/n)}$ ใช้กับ ratio
Amdahl	**$1 / ((1 - P) + P/n)$** โดย P = สัดส่วน**เวลา**
Amdahl max	$1 / (1 - P)$
Single cycle	T_c = critical path (LW = 230 ns ในสไลด์) และ CPI = 1
Multi-cycle (nanoLADA)	**LW 5 · ORI/ORUI/ADD/SW 4 · BEQ/JMP 3**
Pipeline (k stages)	$T = (k + n - 1)\ T_c$ และ speedup → k
Laundry k loads (4 ขั้น × 30 นาที)	(k + 3) / 2 ชั่วโมง
Superscalar (x)	$T = (k + (n - x)/x)\ T_c$
Superpipeline (m)	$T = (k + (n - 1)/m)\ T_c$
Superpipelined superscalar	$T = (k + (n - x)/(m \cdot x))\ T_c$
VLIW (x)	$T = (k + (n/x - x))\ T_c$ (ตามสไลด์) และ IC → n/x
Cache index (ฐานสิบ)	(addr mod cache size) div block size
Cache tag (ฐานสิบ, direct)	addr div cache size
Cache bits	offset = $\log_2$(block B), index = $\log_2$(#sets), tag = ที่เหลือ
จำนวน sets	cache size / (block size × ways)
AMAT	Hit time + Miss rate × Miss penalty
AMAT 2 ระดับ	$H_1 + MR_1 \times (H_2 + MR_2 \times MP_2)$
CPI with cache	CPI_{ideal} + Σ(accesses/instr × miss rate × penalty)
Page table size	2^(VPN bits) × PTE size
Overlap condition	cache size / ways ≤ page size
RAID capacity	R0: N · R1: N/2 · R5: N − 1 · R6: N − 2

เรื่อง	สูตร
2's complement range	$-2^{n-1} \dots 2^{n-1} - 1$
Unsigned range	$0 \dots 2^n - 1$
Full adder	$S = A \oplus B \oplus Cin$ และ $Cout = AB + Cin(A \oplus B)$
FF ขั้นต่ำของ FSM	$\lceil \log_2(\#states) \rceil$
MUX 2^n-to-1	มี n select lines

กับดักรวม (อ่านก่อนเข้าห้องสอบ)

⚠ กับดัก Performance & ISA

1. **Speedup = เวลาเก่า / เวลาใหม่** ห้ามกลับเศษกับส่วน
2. **Amdahl ใช้สัดส่วนของ "เวลา"** ถ้าโจทย์ให้สัดส่วนของ "จำนวนคำสั่ง" + CPI ให้คิดผ่าน CPI แทน
3. **Clock สูงกว่าไม่ได้แปลว่าเร็วกว่า** และ **MIPS สูงกว่าไม่ได้แปลว่าเร็วกว่า** (IC ต่างกันตาม ISA)
4. **GM ทำนายเวลาจริงไม่ได้** แต่ consistent ไม่ว่าจะใช้อะไรเป็น baseline
5. **Little endian = LSB อยู่ address ต่ำ** (Intel) และ **big endian = MSB อยู่ address ต่ำ**
6. **Vector table เก็บ address** (ต้องอ่าน memory อีกครั้ง) ส่วน **handler table เก็บโค้ด** (กระโดดไปได้เลย)

⚠ กับดัก Processor & Pipeline

7. **Multi-cycle ไม่ได้เร็วกว่า single cycle เสมอ** (สไลด์: 4600 vs 4500 ns)
8. **Pipeline ไม่ลด latency** แต่เพิ่ม throughput และ **stage ที่ช้าที่สุด**กำหนด clock
9. **Load-use แก้ด้วย forwarding อย่างเดียวไม่ได้** ต้อง stall 1 cycle หรือ reschedule
10. **ก่อนย้ายคำสั่งเข้า delay slot หรือ reschedule ต้องเช็ค RAW/WAR/WAW**
11. **In-order 5-stage มีแค่ RAW ที่เป็นปัญหาจริง** ส่วน WAR/WAW มาพร้อม OoO หรือ latency ที่ต่างกัน
12. **VLIW ลด IC** ส่วน **superscalar ลด CPI** และ **dynamic scheduling ทำให้ legacy code เร็วขึ้น**

⚠ กับดัก Memory & Storage

13. **Fully associative ไม่มี conflict miss** แต่ยังมี compulsory และ capacity miss
14. **Block ใหญ่ขึ้นไม่ได้ดีเสมอ** (miss penalty สูงขึ้น และถ้าใหญ่เกินไป miss rate กลับเพิ่ม)
15. **Direct-mapped ไม่ต้องมี replacement policy**
16. **Write-back + multiprocessor = incoherent** ส่วน write-through เหมาะกับ multiprocessor กว่า
17. **VM ใช้ write-back เท่านั้น** และ **page fault จัดการโดย OS**
18. **Page table มีต่อ process** ตอน context switch ต้อง flush **TLB** ไม่ใช่ลบ page table
19. **RAID ไม่ใช่ backup** และ **RAID 0 ไม่ทนเสีย**
20. **SAN = block (client จัดการ FS)** ส่วน **NAS = file (server จัดการ FS)**

Glossary A–Z (ค้นหาเร็ว)

คำ	ย่อมาจาก / ชื่อเต็ม	ความหมายสั้น ๆ	บท
ABI	Application Binary Interface	ข้อตกลงระดับ binary: calling convention, stack frame, alignment	1, 3
Accumulator	—	ISA แบบ 1-address มี register หลักตัวเดียวเป็น implicit operand	3
Addressing mode	—	วิธีระบุตำแหน่ง operand (immediate, displacement, indirect, ...)	3
Alignment	—	ข้อมูลขนาด s ต้องเริ่มที่ address ที่ mod s = 0 (restricted)	3
ALU	Arithmetic Logic Unit	วงจรคำนวณ +, −, AND, OR ตาม alu_ops และมี zero flag	0, 4
AMAT	Average Memory Access Time	Hit time + Miss rate × Miss penalty	8
Amdahl's Law	—	speedup รวมถูกจำกัดด้วยส่วนที่ไม่ได้เร่ง	2
Associativity	—	จำนวน way ต่อ set ของ cache (direct = 1 way)	8
Basic block	—	ลำดับคำสั่งที่ไม่มี branch เข้า/ออกตรงกลาง	3, 7
BCD	Binary-Coded Decimal	ใช้ 4 bits ต่อเลขฐานสิบ 1 หลัก	0
Benchmark	—	ชุดโปรแกรมมาตรฐานสำหรับวัด performance	2
Big endian	—	MSB อยู่ address ต่ำ (PowerPC, Sun)	3
Block (cache line)	—	หน่วยข้อมูลที่ย้ายเข้าออก cache ทั้งก้อน	8
Branch delay slot	—	คำสั่งถัดจาก branch ที่ทำงานเสมอ	6
Branch prediction	—	ทาย taken/not taken ล่วงหน้า (history ถูกประมาณ 90%)	6, 7
Bubble	—	ช่องว่างใน pipeline ที่เกิดจาก stall	6
Bus	—	ชุดสายสัญญาณที่ใช้ร่วมกันส่ง data/address/control	1
Cache	—	SRAM เล็กและเร็ว เก็บสำเนาข้อมูลที่ใช้บ่อย จัดการโดย HW	8
Callee/Caller save	—	ใครเป็นคน save register ตอนเรียก subroutine	3
Capacity miss	—	miss เพราะ cache เล็กกว่า working set	8
CIFS / SMB	Common Internet File System / Server Message Block	protocol แชร์ไฟล์ของ Windows (NAS)	9
CISC	Complex Instruction Set Computer	คำสั่งซับซ้อน ความยาวแปรผัน (x86, VAX)	3
CLA	Carry-Lookahead Adder	adder ที่คำนวณ carry ขนานด้วย generate/propagate	0

คำ	ย่อมาจาก / ชื่อเต็ม	ความหมายสั้น ๆ	บท
Clock rate	—	จำนวน cycle ต่อวินาที (Hz)	0, 2
Code motion	—	ย้ายโค้ด loop-invariant ออกนอก loop	3
Compulsory (cold) miss	—	miss ตอนเข้าถึง block ครั้งแรก	8
Conflict miss	—	miss เพราะ mapping ชนกัน ทั้งที่ cache ยังว่าง	8
Control hazard	—	ไม่รู้คำสั่งถัดไปเพราะ branch	6
Control unit	—	decode opcode แล้วสร้าง control signals	4, 5
Copy propagation	—	แทนตัวแปรที่ถูก copy ด้วยต้นฉบับ	3
CPI	Clock (cycles) Per Instruction	จำนวน cycle เฉลี่ยต่อคำสั่ง	2
CPU time	—	$IC \times CPI \times T_c$	2
Critical path	—	path ที่ delay มากที่สุด กำหนด cycle time	0, 4
CSE	Common Subexpression Elimination	คำนวณนิพจน์ซ้ำเพียงครั้งเดียว	3
Data hazard	—	ใช้ข้อมูลก่อนที่มันจะพร้อม (RAW/WAR/WAW)	6
Datapath	—	ส่วนที่เก็บ ย้าย และประมวลผลข้อมูล	4
Decoder	—	n bits → เลือก output 1 จาก 2^n เส้น (one-hot)	0
Deduplication	—	เก็บข้อมูลซ้ำไว้ชุดเดียว	9
Direct-mapped	—	แต่ละ block วางได้ที่เดียวใน cache	8
Dirty bit	—	block ถูกแก้แต่ยังไม่เขียนกลับ (write-back)	8
Displacement	—	addressing แบบ M[R + const] เช่น G.age	3
DRAM	Dynamic RAM	capacitor, ถูก, หนาแน่น, ช้า, ต้อง refresh → main memory	8
Dynamic scheduling	—	HW จัดลำดับคำสั่งตอนรัน (OoO superscalar)	7
EBS	(Amazon) Elastic Block Store	cloud block storage	9
Endian	—	ลำดับ byte ของข้อมูลหลาย byte ใน memory	3
EPC	Exception Program Counter	register เก็บ PC ที่จะกลับไปหลัง exception	5
Exception	—	unprogrammed control flow (interrupt/trap)	3
Extender	—	ขยาย imm 16 → 32 bits (sign/zero extend, zero pad)	4

คำ	ย่อมาจาก / ชื่อเต็ม	ความหมายสั้น ๆ	บท
Fetch	—	IR ← MEM[PC]; PC ← PC + 4	4, 5
FLOPS	Floating-Point Operations Per Second	หน่วยวัดการคำนวณทศนิยม (เชิงการตลาด)	2
Flip-flop	—	หน่วยจำ 1 บิตที่เปลี่ยนค่าที่ขอบ clock	0
Flynn's taxonomy	—	SISD, SIMD, MISD, MIMD	7
Forwarding / Bypassing	—	ส่งผลจาก pipeline register ตรงเข้า ALU	6
FSM	Finite State Machine	วงจร sequential ที่มี state จำกัด และเป็น control ของ multi-cycle	0, 5
Fully associative	—	block วางที่ใดก็ได้ ไม่มี conflict miss	8
Geometric mean	—	ค่าเฉลี่ยของ ratio ที่ consistent กับทุก baseline	2
Gray code	—	รหัสที่ค่าติดกันต่างกัน 1 บิต (แกน K-map)	0
Handler table	—	ตารางที่เก็บโค้ด handler ขนาดคงที่	3
Hazard	—	เหตุที่ pipeline เดินตามจังหวะปกติไม่ได้	6
Hit / Miss rate	—	สัดส่วนที่หาเจอหรือไม่เจอใน cache (miss = 1 – hit)	8
Hyper-Threading	—	ชื่อ SMT ของ Intel	1, 7
IC	Instruction Count	จำนวนคำสั่งที่รันจริง	2
ILP	Instruction-Level Parallelism	จำนวนคำสั่งที่ทำพร้อมกันได้	7
Immediate	—	ค่าคงที่ที่ฝังในคำสั่ง (#10)	3
Index	—	บิตของ address ที่ใช้เลือก set/line ของ cache	8
Interrupt	—	exception จาก hardware ภายนอก	3
IR	Instruction Register	register เก็บคำสั่งที่ fetch มา (multi-cycle)	5
ISA	Instruction Set Architecture	interface ระหว่าง SW กับ HW	1, 3
iSCSI	Internet SCSI	ส่งคำสั่ง block ผ่าน IP (ใช้ทำ SAN)	9
JBOD	Just a Bunch Of Disks	span ดิสก์ต่างขนาด ไม่มี redundancy	9
K-map	Karnaugh map	ตารางช่วยลดรูป Boolean (Gray code, wrap around)	0
Latency	—	เวลาของงานเดี่ยวตั้งแต่เริ่มจนเสร็จ	2, 6
Little endian	—	LSB อยู่ address ต่ำ (Intel)	3
Load-store	—	ISA ที่เข้าถึง memory ผ่าน LD/ST เท่านั้น (RISC)	3

คำ	ย่อมาจาก / ชื่อเต็ม	ความหมายสั้น ๆ	บท
Load-use hazard	—	ใช้ค่าจาก LW ทันที ต้อง stall 1 cycle แม้มี forwarding	6
Locality	—	temporal (เวลา) และ spatial (พื้นที่)	8
Loop unrolling	—	คลี่ loop ให้ basic block ใหญ่ขึ้น	7
LRU	Least Recently Used	ทิ้ง block ที่ไม่ได้ใช้นานที่สุด	8
Mealy / Moore	—	output ขึ้นกับ state + input / state อย่างเดียว	0
Microprogram	—	control แบบเก็บ microinstruction ใน ROM	5
MIMD / MISD	Multiple Instruction Multiple / Single Data	multicore / fault-tolerant	7
MIPS (หน่วย)	Millions of Instructions Per Second	หน่วยเชิงการตลาด เทียบข้าม ISA ไม่ได้	2
Miss penalty	—	เวลานำ block เข้า cache เมื่อ miss	8
Moore's Law	—	transistor เพิ่ม 2 เท่าทุก ~2 ปี (ข้อสังเกต)	1
MSB / LSB	Most / Least Significant Bit	บิตซ้ายสุด / ขวาสุด	0
MUX	Multiplexer	เลือก 1 จาก 2^n อินพุตด้วย n select bits	0, 4
nanoLADA	—	RISC ISA ที่ใช้สอนในวิชา (7 คำสั่ง, 3 formats)	4
NAS	Network Attached Storage	file storage ผ่าน network (NFS, SMB)	9
NFS	Network File System	protocol แชร์ไฟล์ (Unix)	9
NOP	No OPeration	คำสั่งที่ไม่ทำอะไร ใช้เว้นระยะ	6
Object storage	—	เก็บเป็น object ผ่าน API (S3)	9
Offset	—	บิตล่างของ address ที่เลือก byte ใน block/page	8
OoO	Out-of-Order execution	ทำคำสั่งที่ operand พร้อมก่อน	7
Opcode	Operation code	field บอกชนิดคำสั่ง (6 bits ใน nanoLADA)	3, 4
Organization	Computer Organization	โครงสร้างภายในที่ implement ISA	1
Overflow	—	ผลเกินช่วง signed (บวก + บวก ได้ลบ)	0
Page / Page frame	—	block ของ VM ฝั่ง virtual / physical	8
Page fault	—	miss ของ VM ที่ OS จัดการ	8
Page table	—	ตาราง VPN → PPN ต่อ process	8
Parity	—	ข้อมูล XOR สำหรับกู้ดิสก์ที่เสีย	9

คำ	ย่อมาจาก / ชื่อเต็ม	ความหมายสั้น ๆ	บท
PC	Program Counter	register เก็บ address ของคำสั่งถัดไป	3, 4
PC-relative	—	branch target = PC + displacement (relocatable)	3
Pipeline	—	ทำหลายคำสั่งซ้อนกันคนละ stage	6
PLA / PAL	Programmable Logic Array / Array Logic	AND–OR plane ที่ program ได้ ใช้ทำ control	0, 4
RAID	Redundant Array of Independent Disks	รวมดิสก์เพื่อ performance/redundancy	9
RAM	Random Access Memory	เวลาเข้าถึงคงที่ทุกตำแหน่ง	8
RAW / WAR / WAW	Read After Write / Write After Read / Write After Write	ชนิดของ data hazard	6
Register file	—	32 × 32-bit, อ่าน 2 port, เขียน 1 port	4
Register renaming	—	ใช้ physical register ต่างกันเพื่อลบ WAR/WAW	7
Response time	—	เวลาของงานหนึ่งงาน (= latency)	2
RISC	Reduced Instruction Set Computer	คำสั่งพื้นฐาน, fixed length, load-store	3
ROM	Read-Only Memory	เก็บ truth table หรือ microprogram	0, 5
RTL	Register Transfer Language	สัญกรณ์อธิบายการย้ายข้อมูลของคำสั่ง	4, 5
S3	(Amazon) Simple Storage Service	cloud object storage	9
SAN	Storage Area Network	block storage ผ่าน network (iSCSI)	9
SAS / SATA	Serial Attached SCSI / Serial ATA	interface ของ storage	9
Set-associative	—	block วางได้ใน n ways ของ set ตัวเอง	8
Sign extension	—	ขยายบิตโดยคัดลอก sign bit	0, 4
SIMD / SISD	Single Instruction Multiple / Single Data	vector processor / uniprocessor	7
SMT	Simultaneous MultiThreading	ดึงคำสั่งจากหลาย thread เข้า pipeline	7
Software pipelining	—	จัด loop ให้ทำคนละขั้นของคนละ iteration	7

คำ	ย่อมาจาก / ชื่อเต็ม	ความหมายสั้น ๆ	บท
SPEC	Standard Performance Evaluation Corporation	องค์กร benchmark มาตรฐาน	2
Speculative execution	—	ทำงานล่วงหน้าก่อนรู้แน่ว่าต้องทำ	7
Speedup	—	T_{old} / T_{new}	2
SRAM	Static RAM	transistor, เร็ว, แพง, ไม่ต้อง refresh → cache	8
SSD / HDD	Solid State / Hard Disk Drive	flash / จานแม่เหล็ก	9
Stack architecture	—	ISA แบบ 0-address ใช้ top of stack	3
Stall	—	หยุดคำสั่งใน pipeline รอ	6
Static scheduling	—	compiler จัดลำดับคำสั่ง (VLIW)	7
Strength reduction	—	ใช้ operation ที่ถูกกว่า เช่น shift แทนคูณ	3
Striping / Mirroring	—	กระจายข้อมูล / ทำสำเนา (RAID 0 / RAID 1)	9
Structural hazard	—	resource ไม่พอ ถูกแย่งใช้พร้อมกัน	6
Superpipeline	—	แบ่ง cycle เป็น m minor cycles	7
Superscalar	—	หลาย pipeline, HW issue หลายคำสั่งต่อ cycle	7
Tag	—	บิตบนของ address เก็บใน cache ไว้ตรวจ hit	8
Throughput	—	จำนวนงานที่เสร็จต่อหน่วยเวลา	2, 6
TLB	Translation Lookaside Buffer	cache ของ page table entries	8
TLP	Thread-Level Parallelism	ความขนานระดับ thread	7
Trap	—	exception จาก software	3
2's complement	Two's complement	$-X$ = กลับบิต + 1	0
Valid bit	—	line นี้มีข้อมูลที่ใช้ได้หรือยัง	8
Vector table	—	ตารางที่เก็บ address ของ handler	3
Virtual memory	—	address space เสมือนต่อ process (cost, relocation, protection)	8
VLIW	Very Long Instruction Word	compiler รวมหลาย op ใน 1 คำสั่ง (static)	7
VPN / PPN	Virtual / Physical Page Number	ส่วนบนของ VA / PA	8
Write allocate	—	write miss แล้วโหลด block มาก่อนเขียน	8

คำ	ย่อมาจาก / ชื่อเต็ม	ความหมายสั้น ๆ	บท
Write-back / Write-through	—	เขียน cache แล้วค่อยเขียนกลับ / เขียนทั้งสองที่ทันที	8
Write buffer	—	FIFO พักการเขียนของ write-through	8
XOR	Exclusive OR	เป็น 1 เมื่ออินพุตต่างกัน	0, 3